

Applied Forecasting

Problem Set 6: Time Series

Shadad Hossain

2025-04-21

Question 1 Olympic Running:

```
## Warning: package 'fpp3' was built under R version 4.3.3

## Registered S3 method overwritten by 'tsibble':
##   method           from
##   as_tibble.grouped_df dplyr

## -- Attaching packages ----- fpp3 1.0.1 --

## v tibble      3.2.1      v tsibble      1.1.6
## v dplyr       1.1.4      v tsibbledata 0.4.1
## v tidyr       1.3.1      v feasts      0.4.1
## v lubridate   1.9.3      v fable       0.4.1
## v ggplot2     3.5.1

## Warning: package 'tsibble' was built under R version 4.3.3

## Warning: package 'feasts' was built under R version 4.3.3

## Warning: package 'fabletools' was built under R version 4.3.3

## Warning: package 'fable' was built under R version 4.3.3

## -- Conflicts ----- fpp3_conflicts --
## x lubridate::date()      masks base::date()
## x dplyr::filter()        masks stats::filter()
## x tsibble::intersect()   masks base::intersect()
## x tsibble::interval()   masks lubridate::interval()
## x dplyr::lag()           masks stats::lag()
## x tsibble::setdiff()     masks base::setdiff()
## x tsibble::union()       masks base::union()

## Warning: package 'gt' was built under R version 4.3.3

## -- Attaching core tidyverse packages ----- tidyverse 2.0.0 --
## v forcats 1.0.0      v stringr 1.5.1
## v readr    2.1.5
```

```
## -- Conflicts ----- tidyverse_conflicts() --
## x dplyr::filter()      masks stats::filter()
## x tsibble::interval() masks lubridate::interval()
## x dplyr::lag()         masks stats::lag()
## i Use the conflicted package (<http://conflicted.r-lib.org/>) to force all conflicts to become errors
```

Part 1.1 Plot the winning time against the year for each event. Describe the main features of the plot.

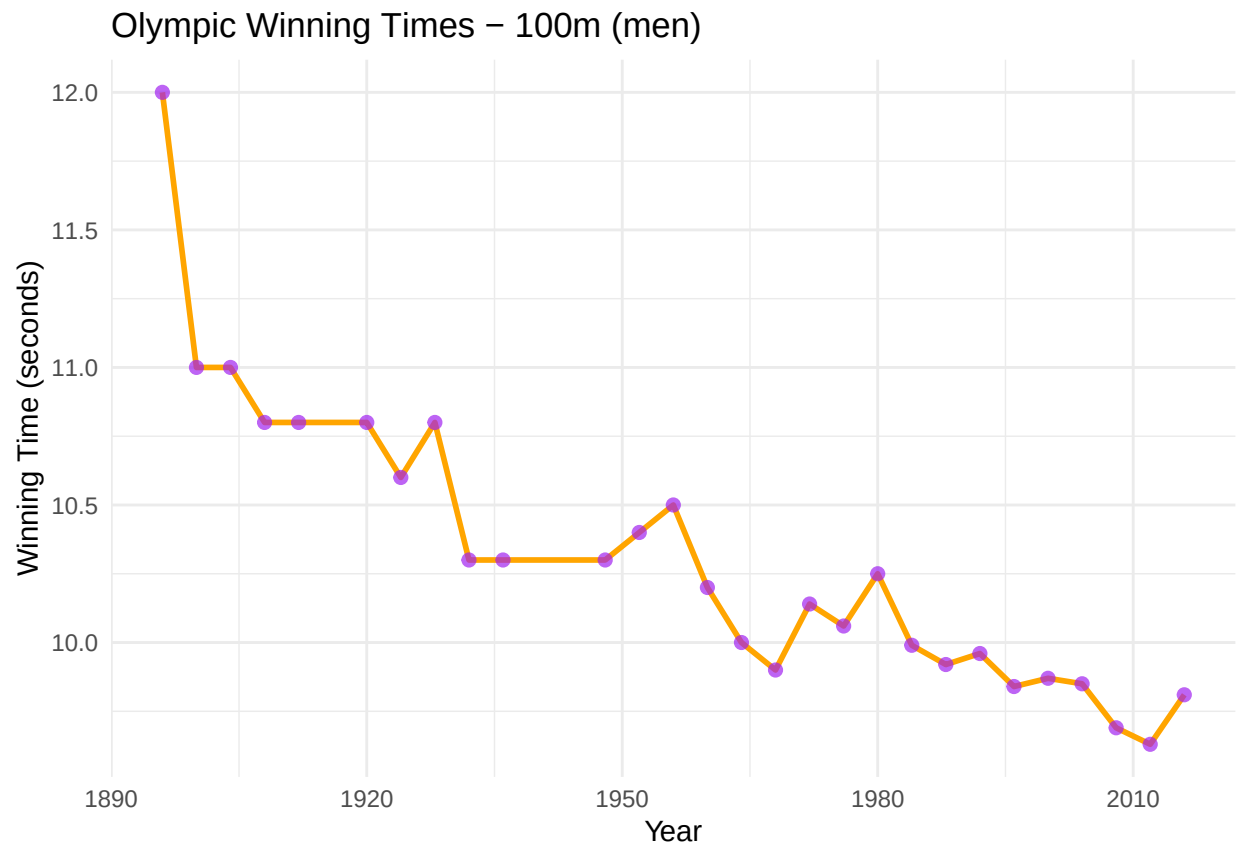
```
# Clean and prepare the data
olympic_clean <- olympic_running %>%
  as_tibble() %>%
  filter(!is.na(Time)) # drop rows with missing times

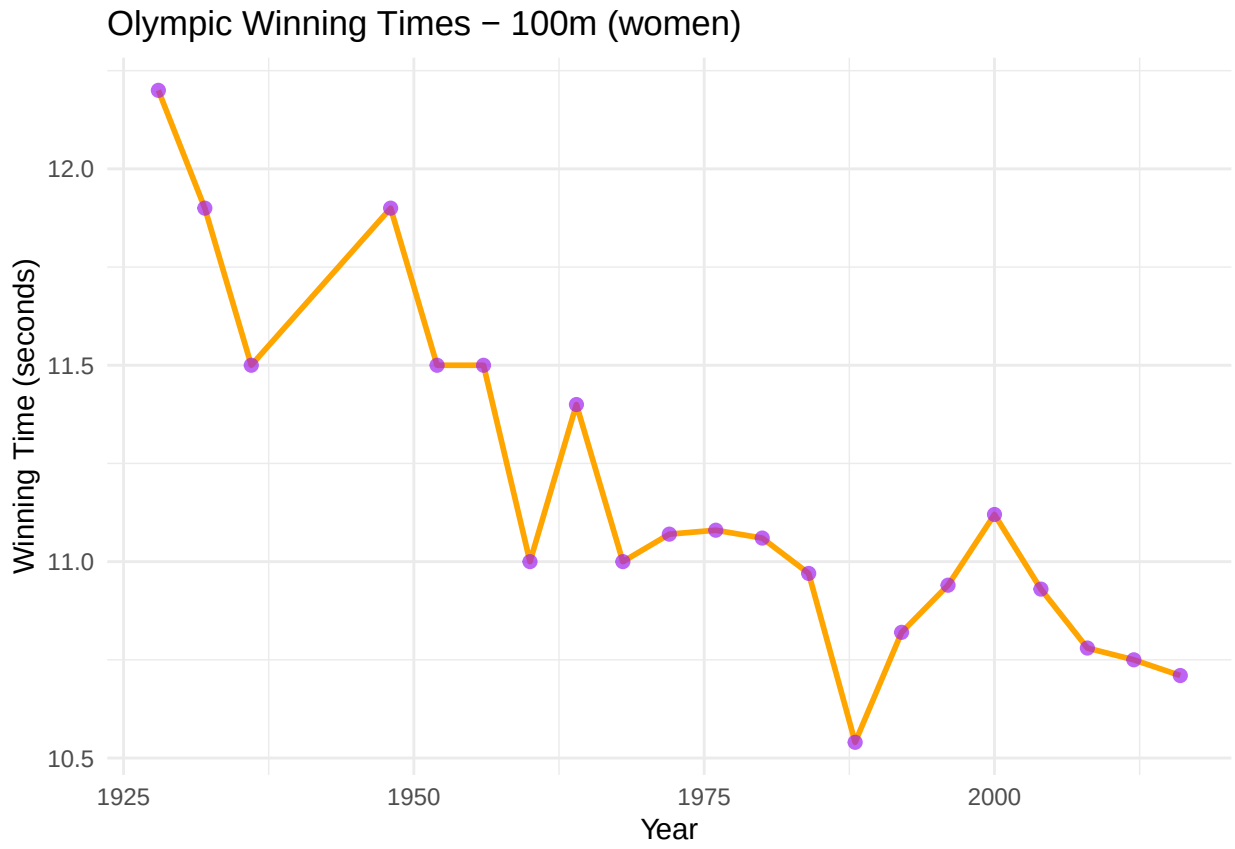
# Create unique combinations of Length and Sex
event_gender_combos <- olympic_clean %>%
  distinct(Length, Sex)

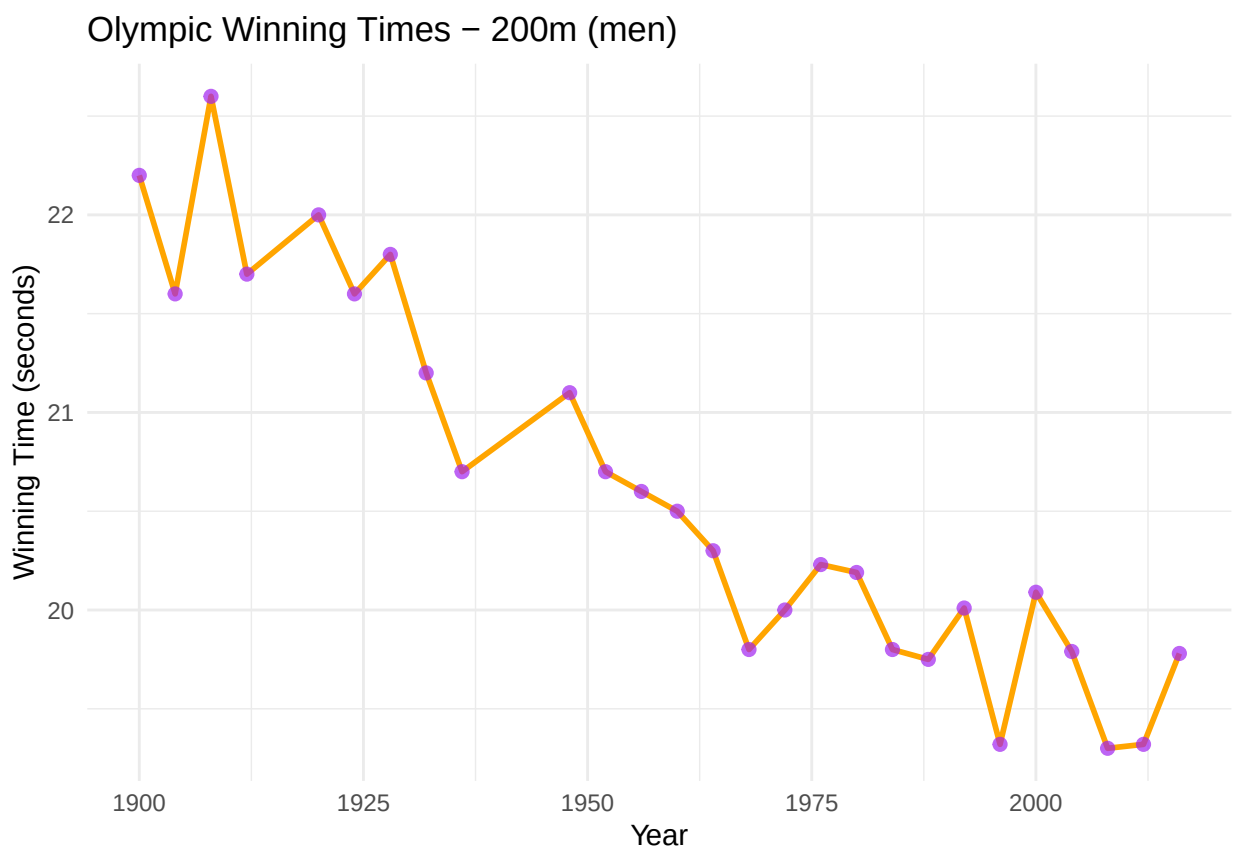
# Loop through each (Length, Sex) combo and plot
for (i in 1:nrow(event_gender_combos)) {
  dist <- event_gender_combos$Length[i]
  gender <- event_gender_combos$Sex[i]

  p <- olympic_clean %>%
    filter(Length == dist, Sex == gender) %>%
    ggplot(aes(x = Year, y = Time)) +
    geom_line(color = "orange", linewidth = 1) +
    geom_point(color = "purple", size = 2, alpha = 0.7) +
    labs(
      title = paste0("Olympic Winning Times - ", dist, "m (", gender, ")"),
      x = "Year",
      y = "Winning Time (seconds)"
    ) +
    theme_minimal()

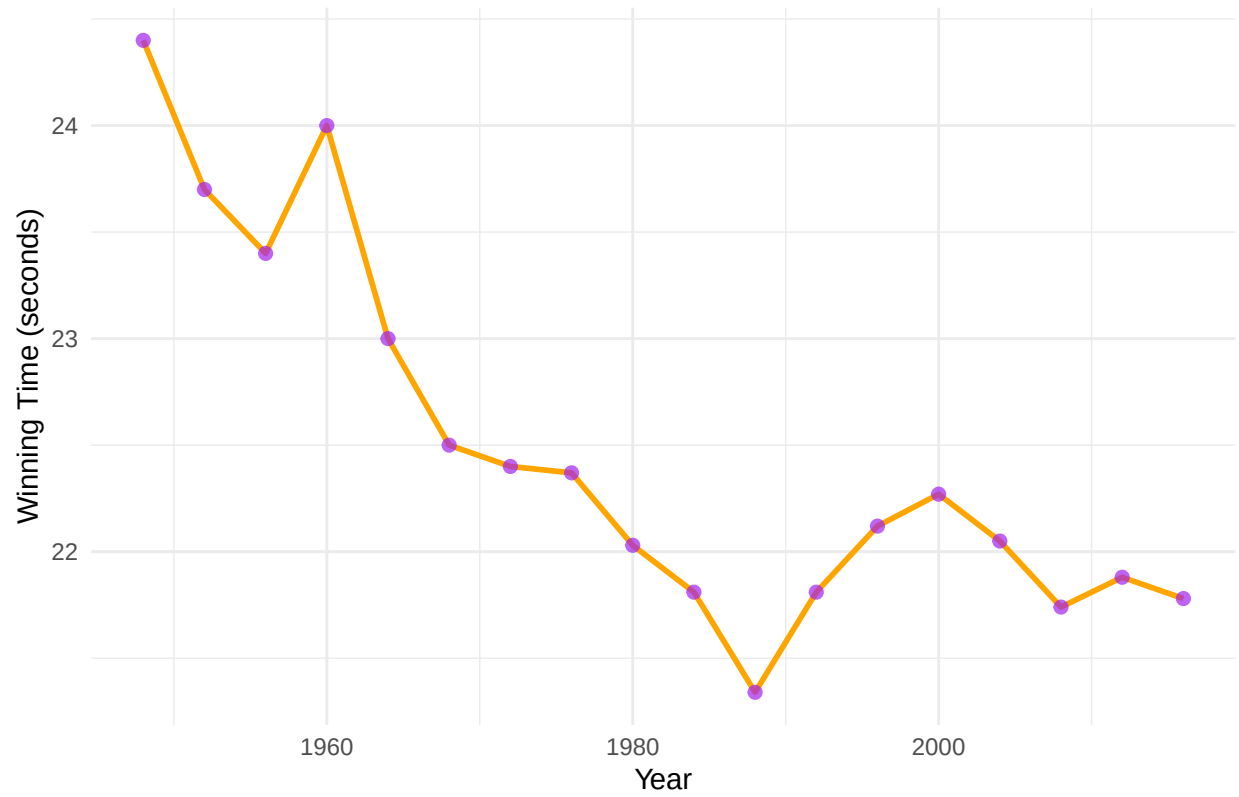
  print(p)
}
```



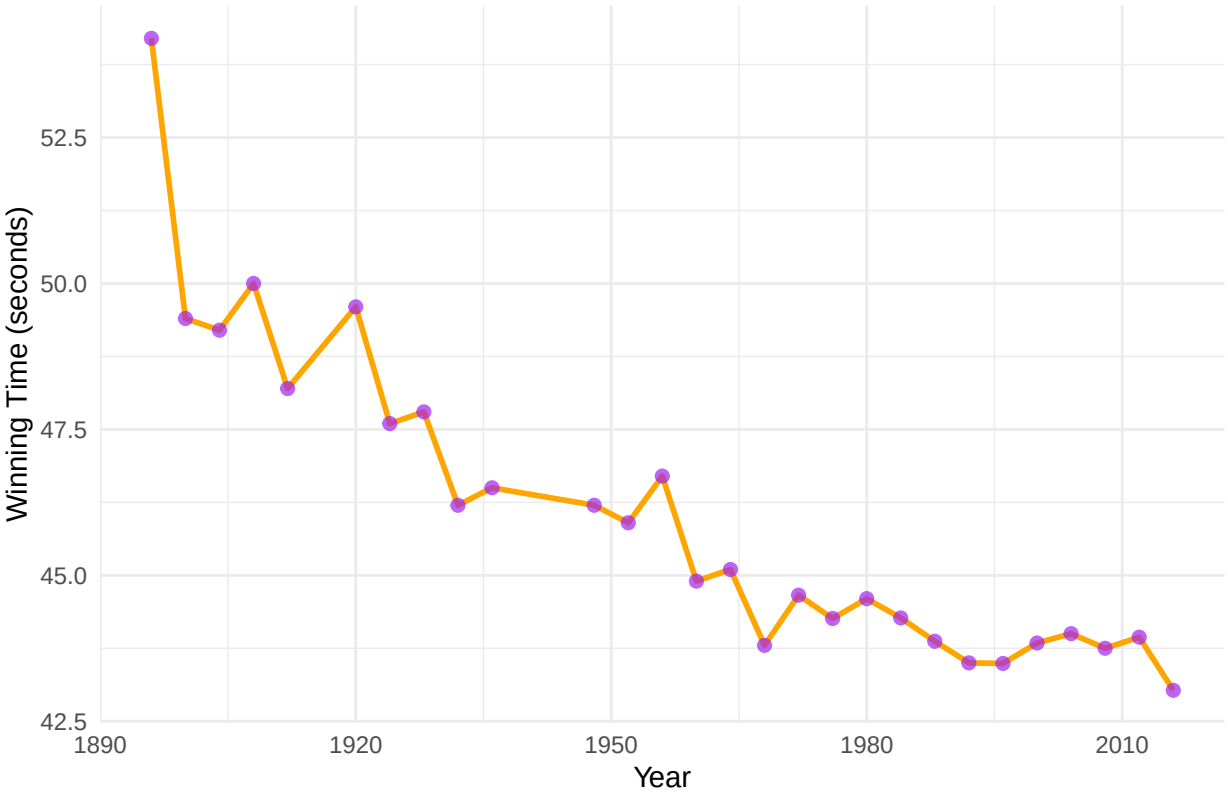


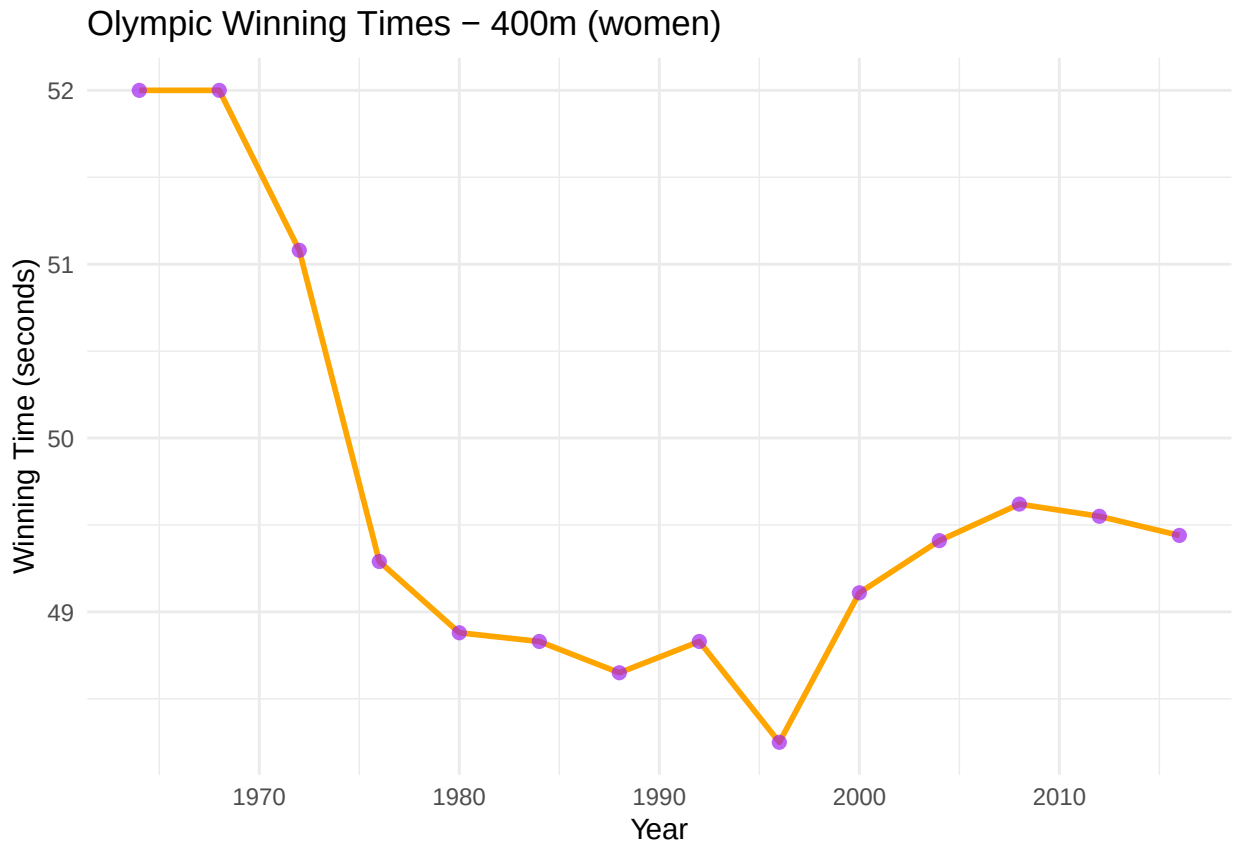


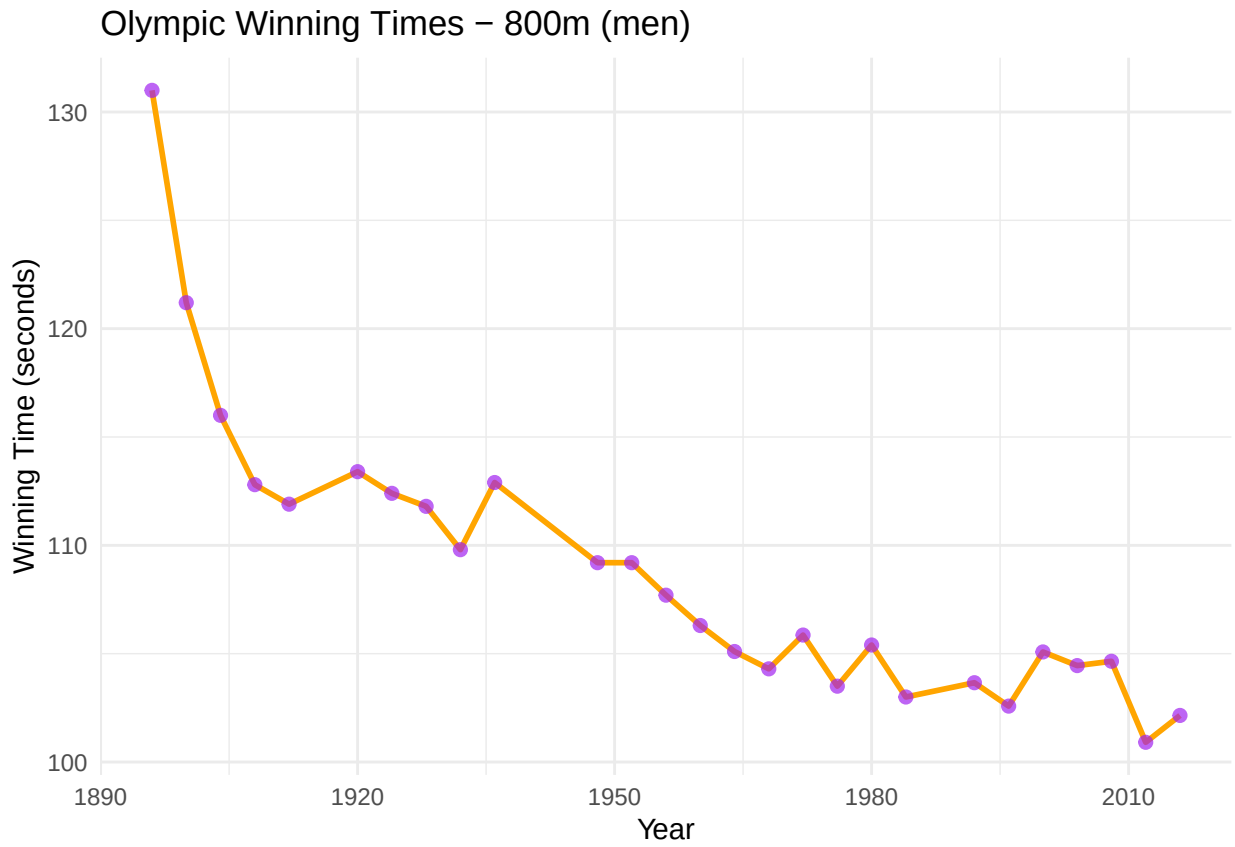
Olympic Winning Times – 200m (women)

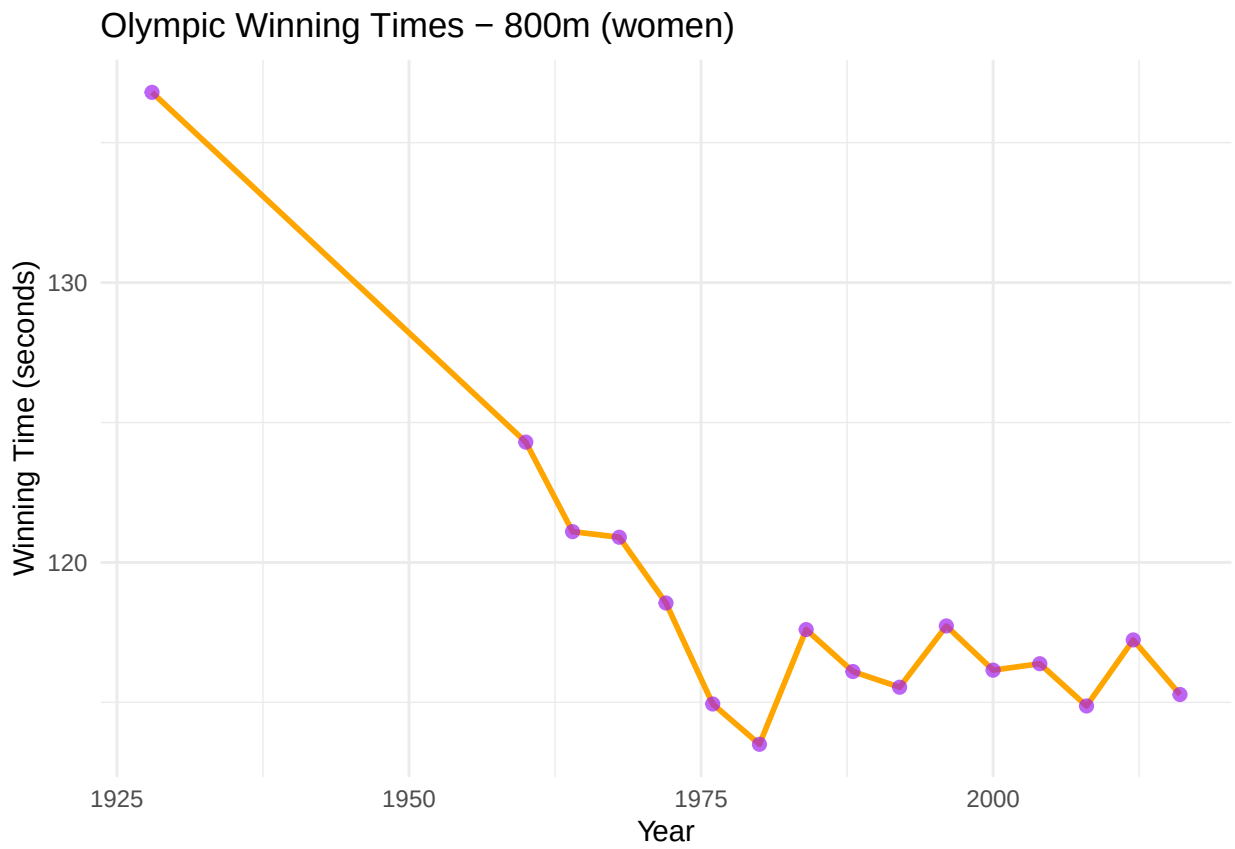


Olympic Winning Times – 400m (men)

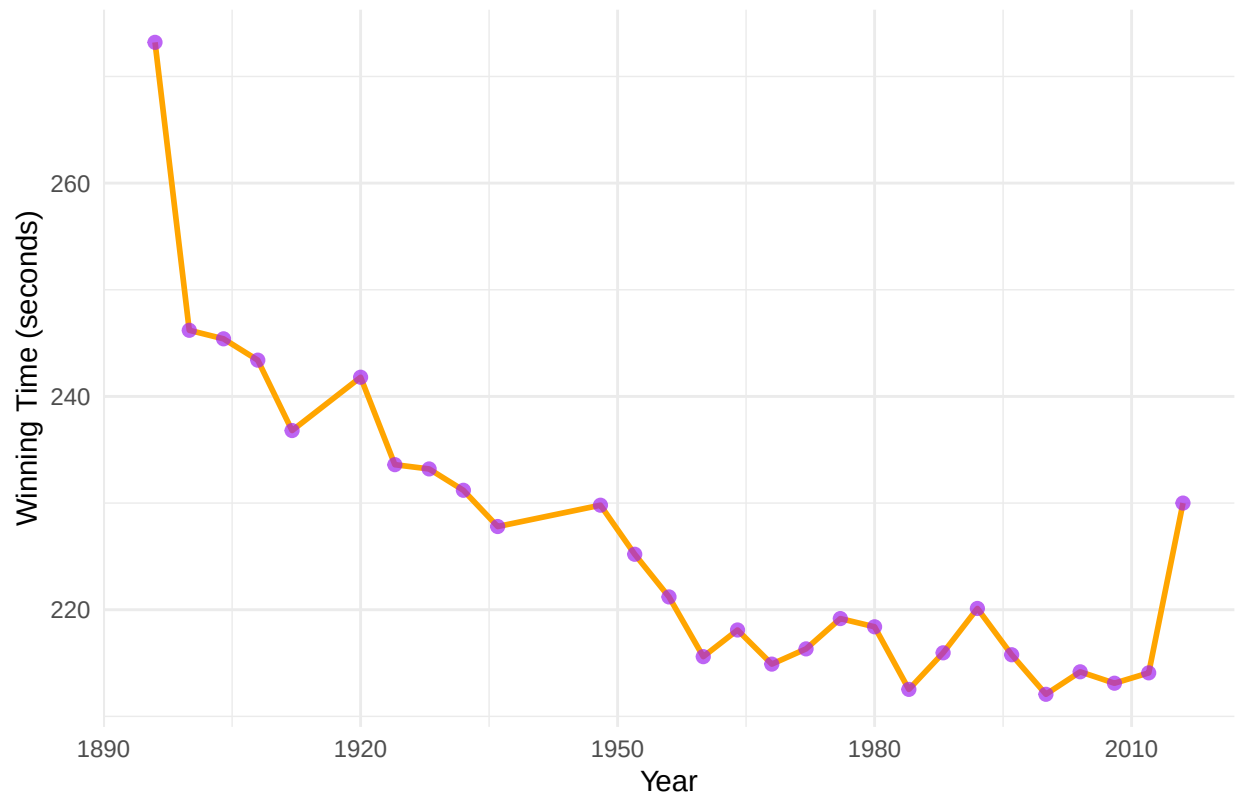


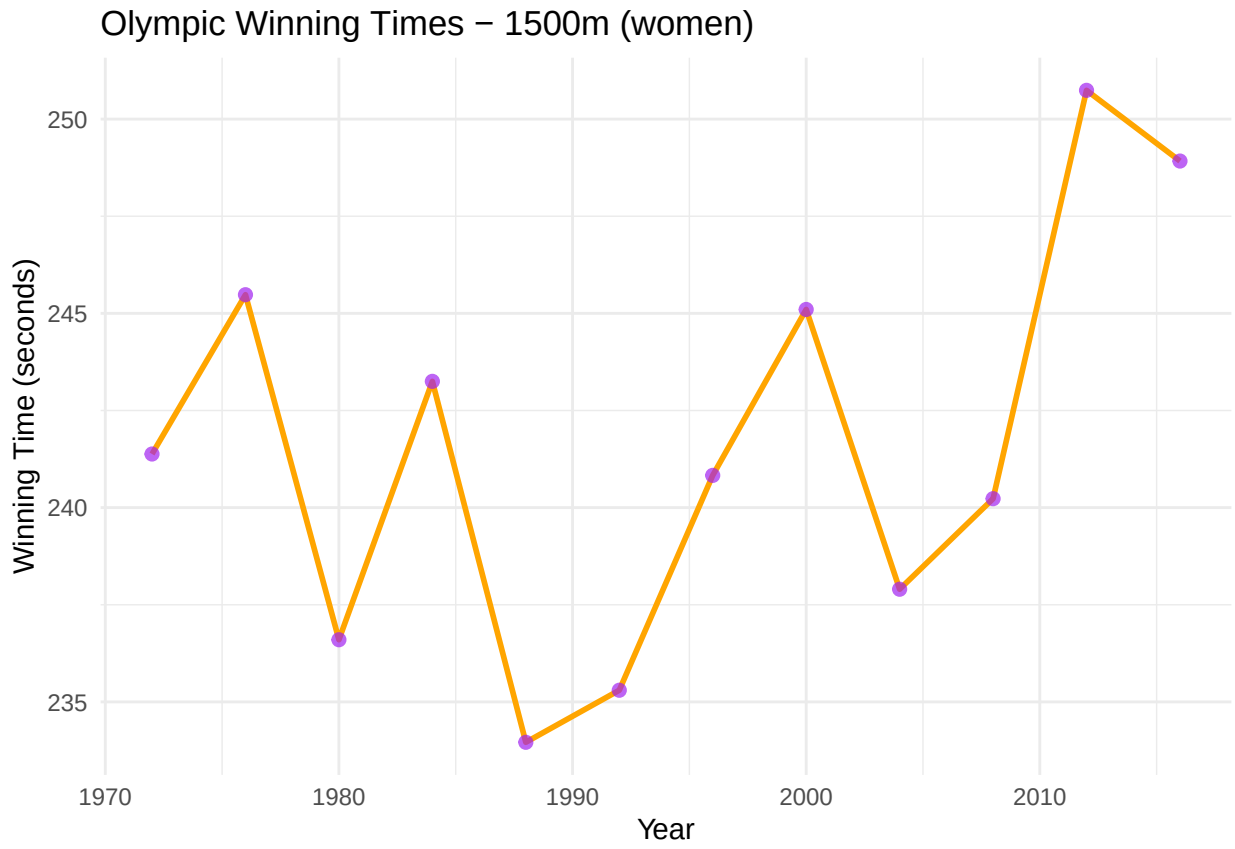


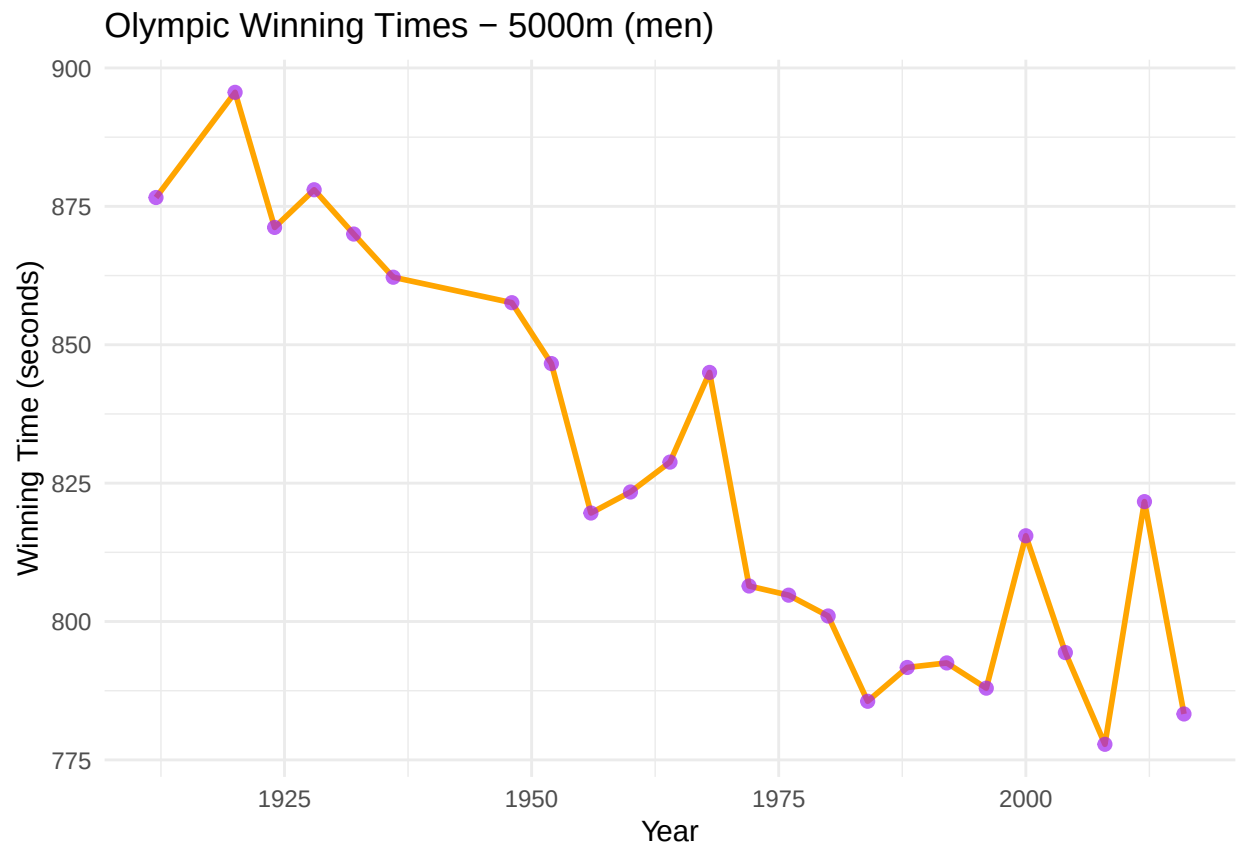


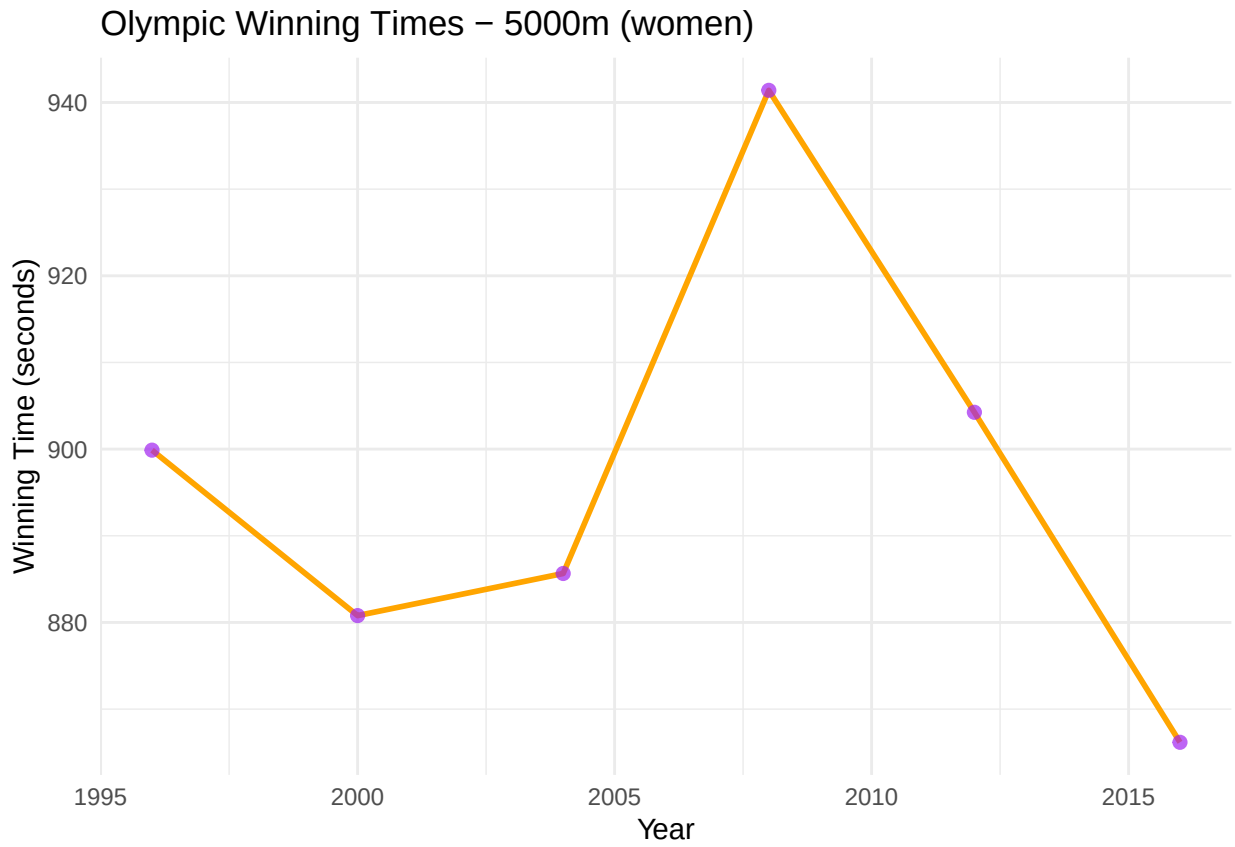


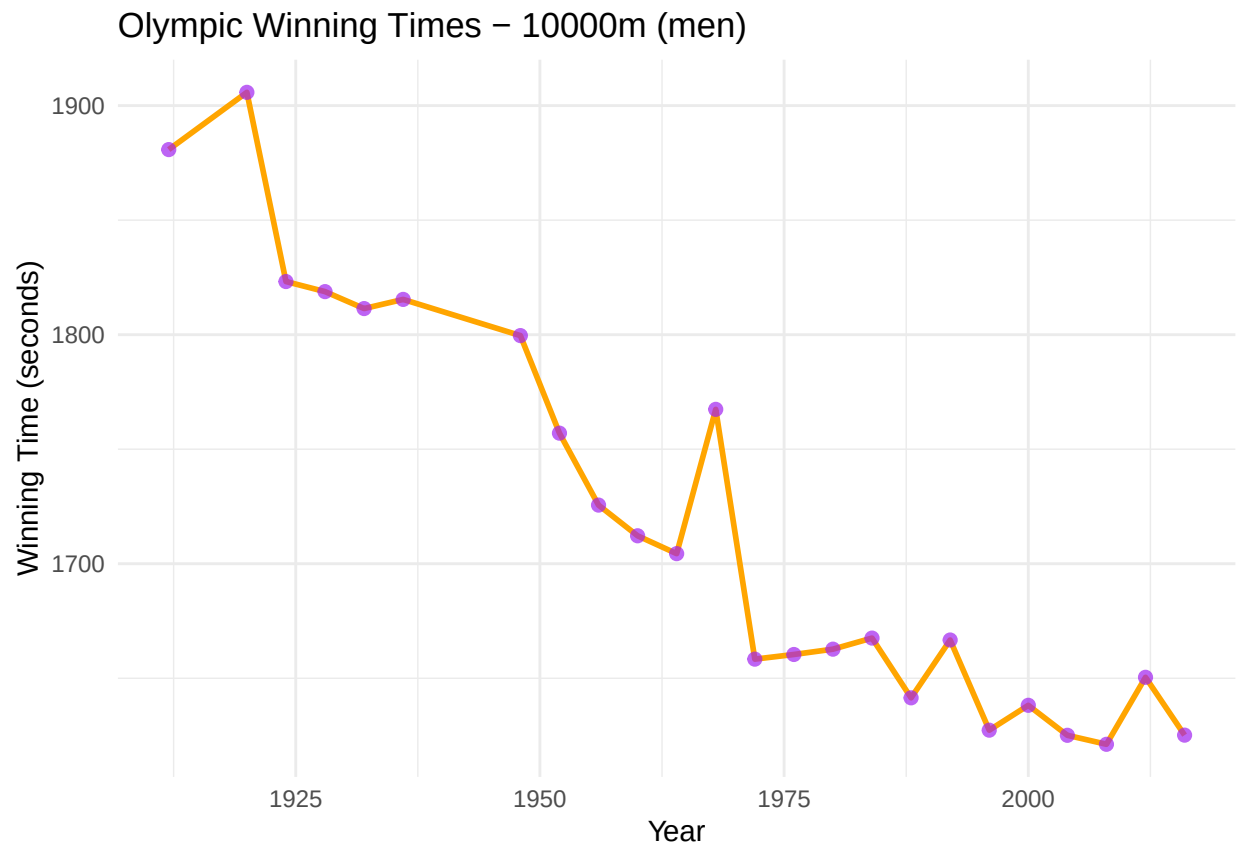
Olympic Winning Times – 1500m (men)

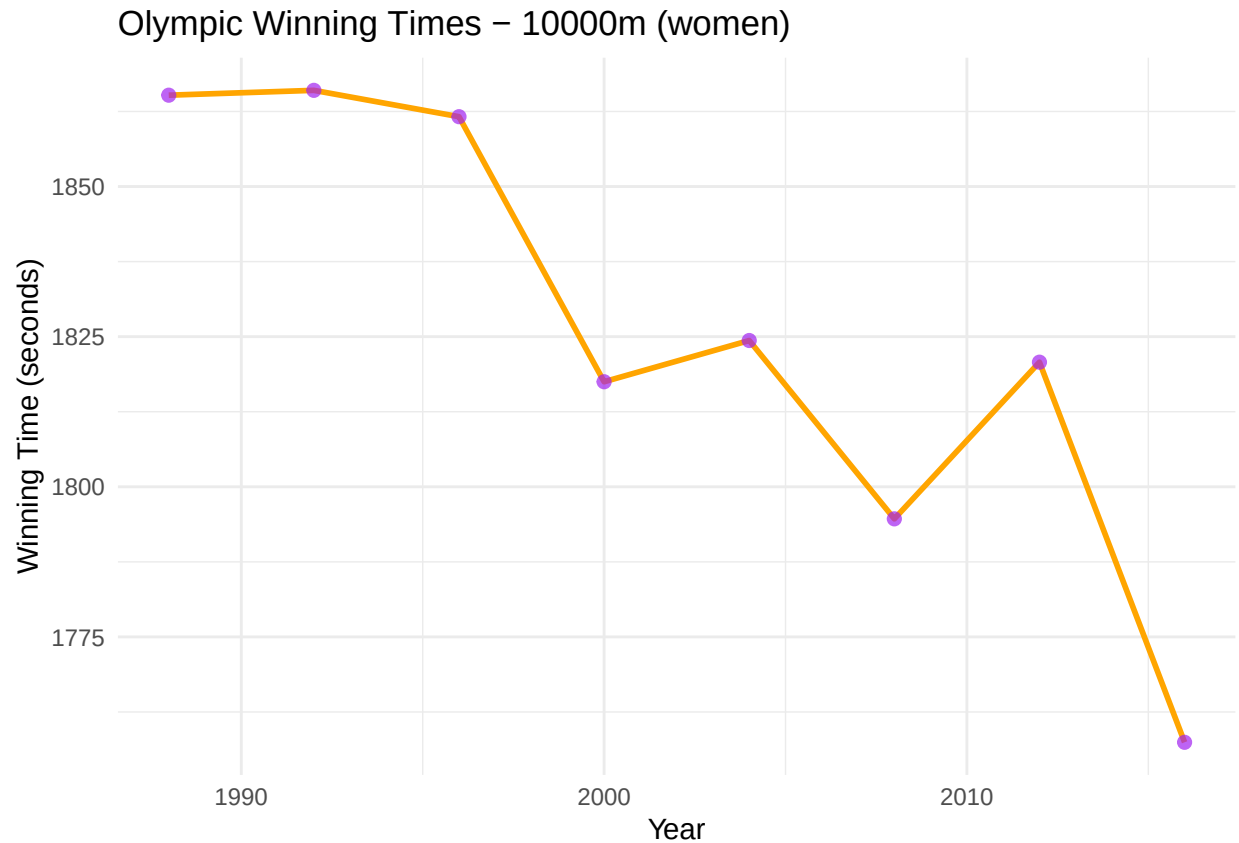












Key Features:

Men's Events:

- Clear downward trends in winning times across all distances, indicating improved performance over time.
- Most events show rapid improvement in earlier periods (pre-1970s) followed by a gradual flattening in recent decades.
- Shorter distances (100m, 200m) show more consistent improvements with less variability.
- Longer distances (5000m, 10000m) display more year-to-year variability but maintain overall downward trends.
- Several events show notable “steps” in improvement rather than smooth progression.
- The rate of improvement appears to be slowing in recent decades across most events.

Women's Events:

- Women's events generally have shorter time series as they were introduced to the Olympics later.
- Similar to men's events, most show clear improvement over time.
- Some events (particularly 800m and 1500m) show more variability and less clear trends in recent years.
- The 5000m and 10000m women's events have the shortest histories and most variable patterns.

Part 1.2 Fit a regression line to the data for each event. Obviously the winning times have been decreasing, but at what average rate per year?

```
# Fit regression model Time ~ Year for each Length + Sex
regression_results <- olympic_clean %>%
  group_by(Length, Sex) %>%
  summarise(
    model = list(lm(Time ~ Year, data = cur_data())),
    .groups = "drop"
  ) %>%
  mutate(
    Slope = map_dbl(model, ~ coef(.x)["Year"]),
    Intercept = map_dbl(model, ~ coef(.x)["(Intercept)"])
  ) %>%
  arrange(Sex, Length)

## Warning: There was 1 warning in 'summarise()'.
## i In argument: 'model = list(lm(Time ~ Year, data = cur_data()))'.
## i In group 1: 'Length = 100' 'Sex = "men"'.
## Caused by warning:
## ! 'cur_data()' was deprecated in dplyr 1.1.0.
## i Please use 'pick()' instead.
```

```
# Format output table nicely
regression_results %>%
  select(Length, Sex, Slope) %>%
  mutate(
    Slope = round(Slope, 4),
    Interpretation = paste0(abs(Slope), " sec/year decrease")
  ) %>%
  gt() %>%
  tab_header(
    title = "Average Yearly Change in Olympic Winning Times",
    subtitle = "Linear regression slope (Time ~ Year) for each event"
  ) %>%
  cols_label(
    Length = "Distance (m)",
    Sex = "Gender",
    Slope = "Slope",
    Interpretation = "Interpretation"
  )
```

Analysis of Rate of Decrease:

- The negative slopes confirm that winning times have been decreasing over the years across all events.
- For men's events, the rate of improvement (in seconds per year) increases with race length: from approximately -0.01 seconds per year for the 100m sprint to over -2.0 seconds per year for the 10000m.
- Women's events follow a similar pattern, with longer distances showing larger yearly time improvements.
- The absolute magnitude of improvement correlates strongly with race duration, which is expected since longer races have more room for improvement.
- The relative improvement rates (as a percentage of total race time) are more comparable across events.

Average Yearly Change in Olympic Winning Times

Linear regression slope (Time ~ Year) for each event

Distance (m)	Gender	Slope	Interpretation
100	men	-0.0126	0.0126 sec/year decrease
200	men	-0.0249	0.0249 sec/year decrease
400	men	-0.0646	0.0646 sec/year decrease
800	men	-0.1518	0.1518 sec/year decrease
1500	men	-0.3151	0.3151 sec/year decrease
5000	men	-1.0299	1.0299 sec/year decrease
10000	men	-2.6659	2.6659 sec/year decrease
100	women	-0.0142	0.0142 sec/year decrease
200	women	-0.0336	0.0336 sec/year decrease
400	women	-0.0401	0.0401 sec/year decrease
800	women	-0.1980	0.198 sec/year decrease
1500	women	0.1468	0.1468 sec/year decrease
5000	women	-0.3030	0.303 sec/year decrease
10000	women	-3.4962	3.4962 sec/year decrease

Part 1.3 Plot the residuals against the year. What does this indicate about the suitability of the fitted lines?

```
# Loop: Fit model, compute residuals, plot
for (i in 1:nrow(event_gender_combos)) {
  dist <- event_gender_combos$Length[i]
  gender <- event_gender_combos$Sex[i]

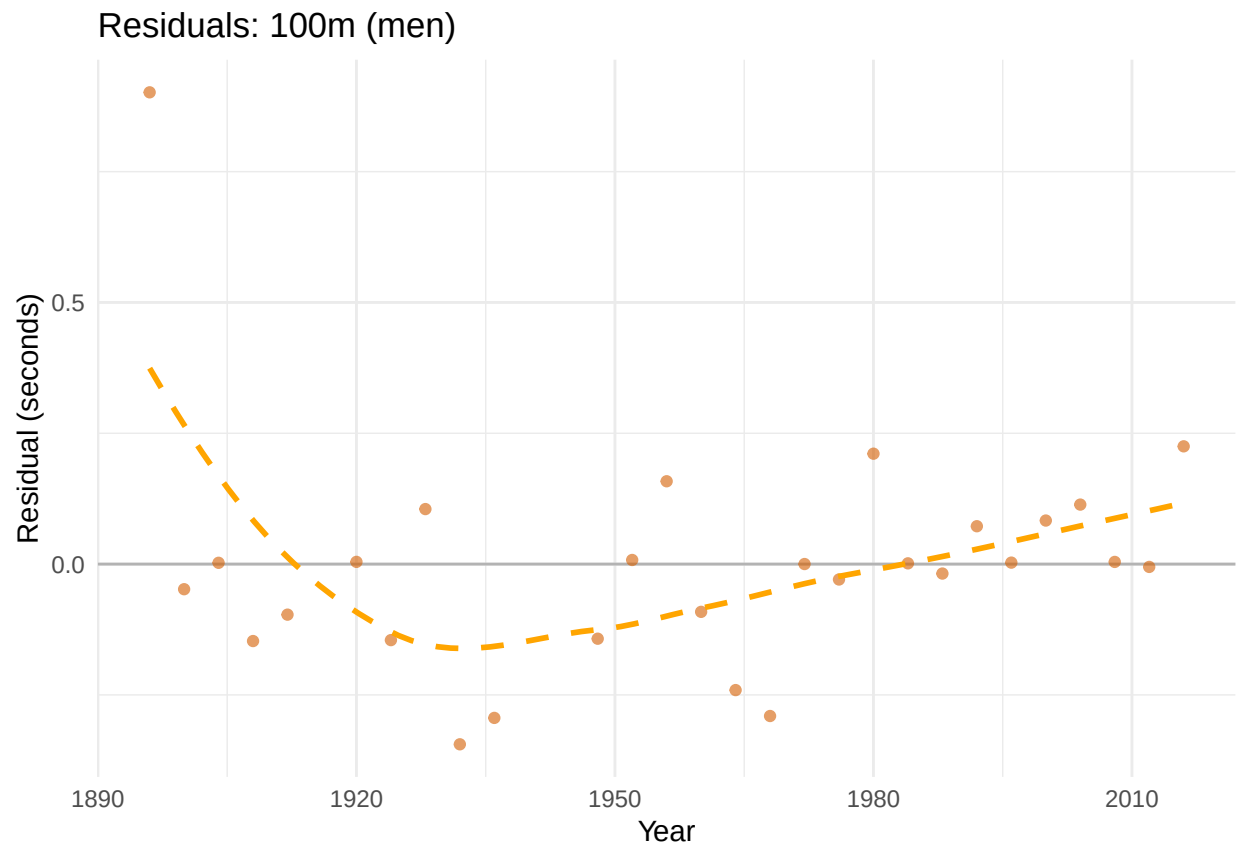
  df <- olympic_clean %>%
    filter(Length == dist, Sex == gender)

  model <- lm(Time ~ Year, data = df)
  df$residual <- resid(model)

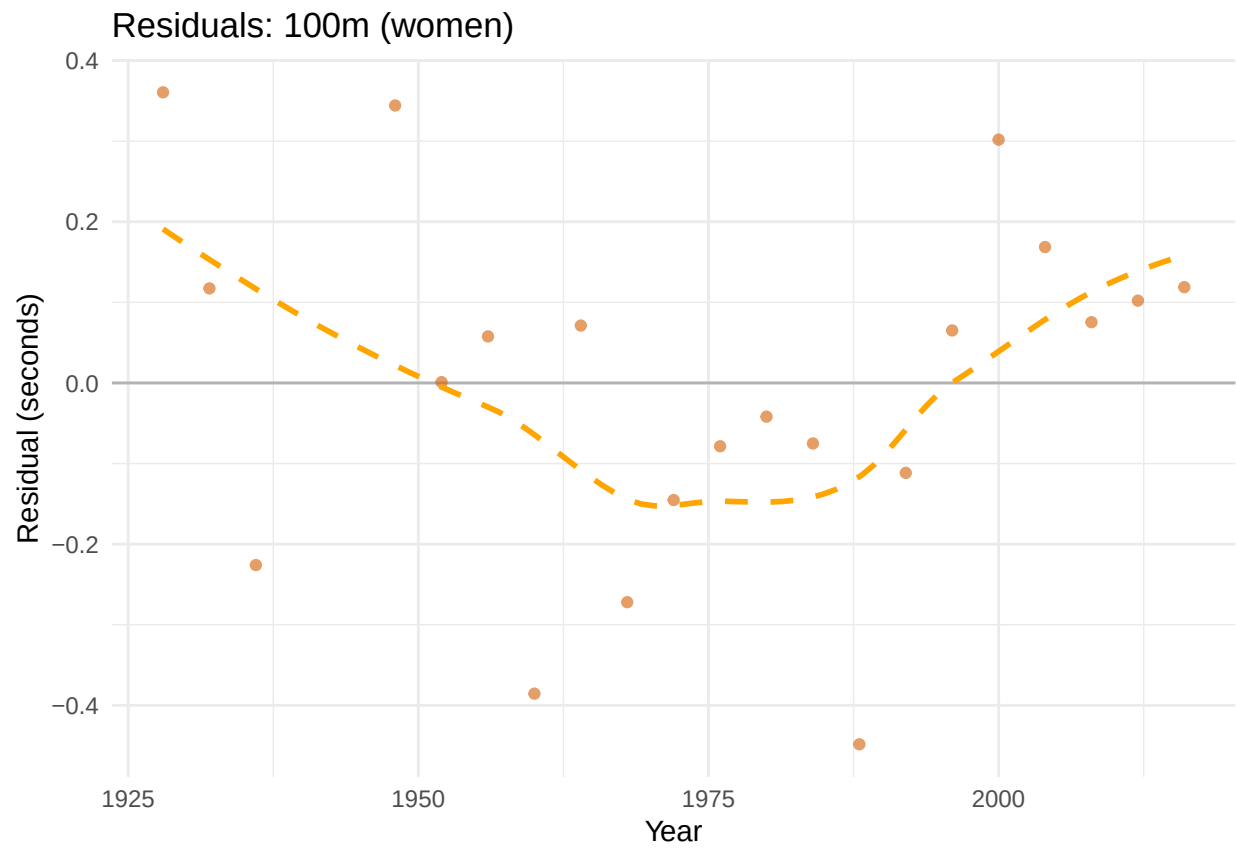
  p <- ggplot(df, aes(x = Year, y = residual)) +
    geom_hline(yintercept = 0, color = "gray70") +
    geom_point(alpha = 0.6, color = "#D55E00") +
    geom_smooth(method = "loess", se = FALSE, color = "orange", linetype = "dashed") +
    labs(
      title = paste0("Residuals: ", dist, "m (", gender, ")"),
      x = "Year",
      y = "Residual (seconds)"
    ) +
    theme_minimal()

  print(p)
}
```

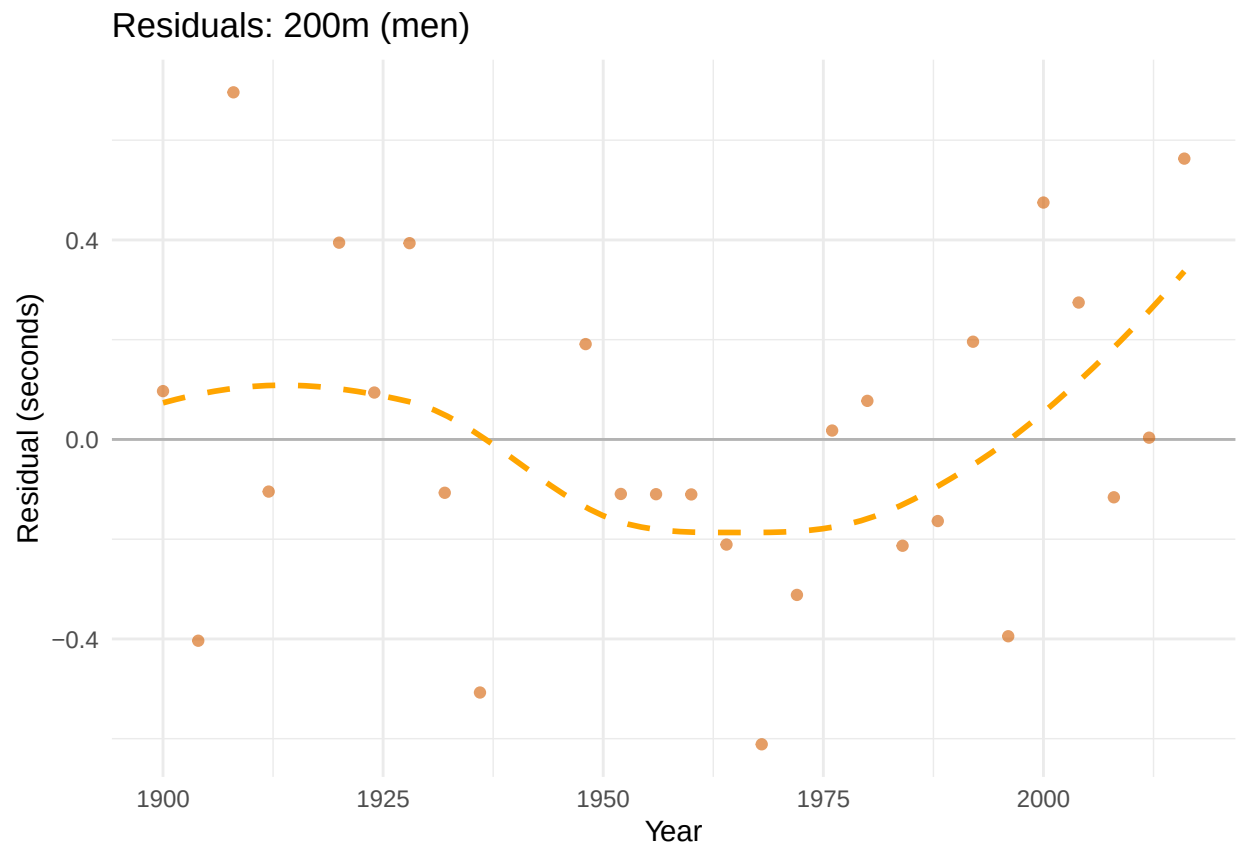
```
## 'geom_smooth()' using formula = 'y ~ x'
```



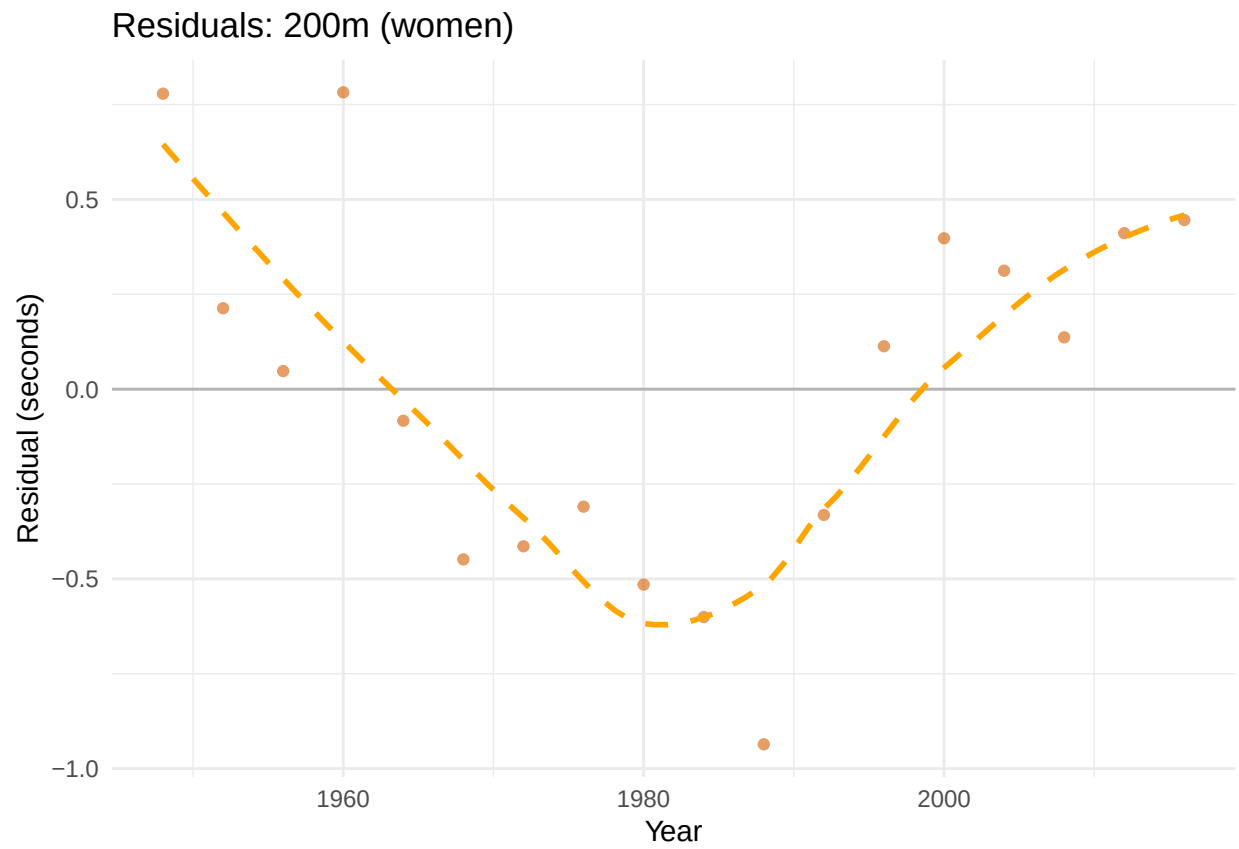
```
## 'geom_smooth()' using formula = 'y ~ x'
```



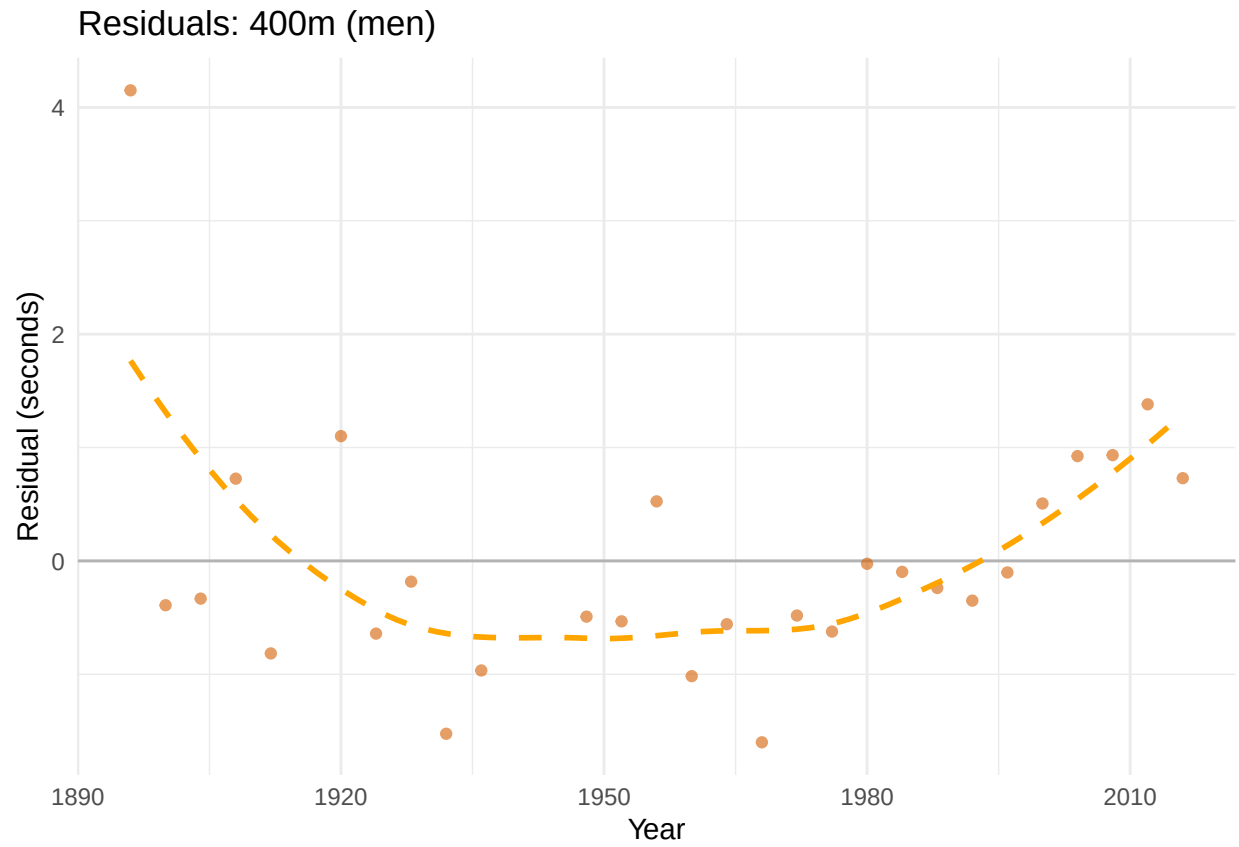
```
## 'geom_smooth()' using formula = 'y ~ x'
```



```
## 'geom_smooth()' using formula = 'y ~ x'
```

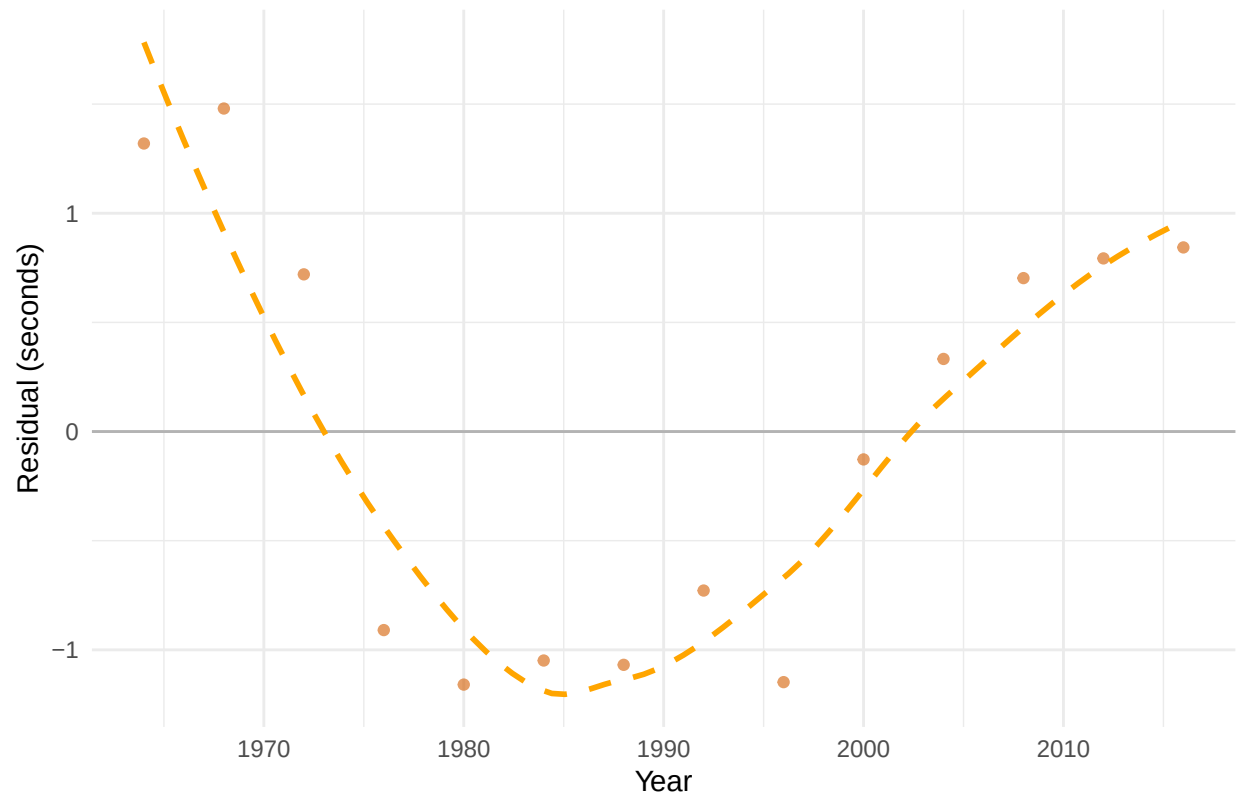


```
## 'geom_smooth()' using formula = 'y ~ x'
```

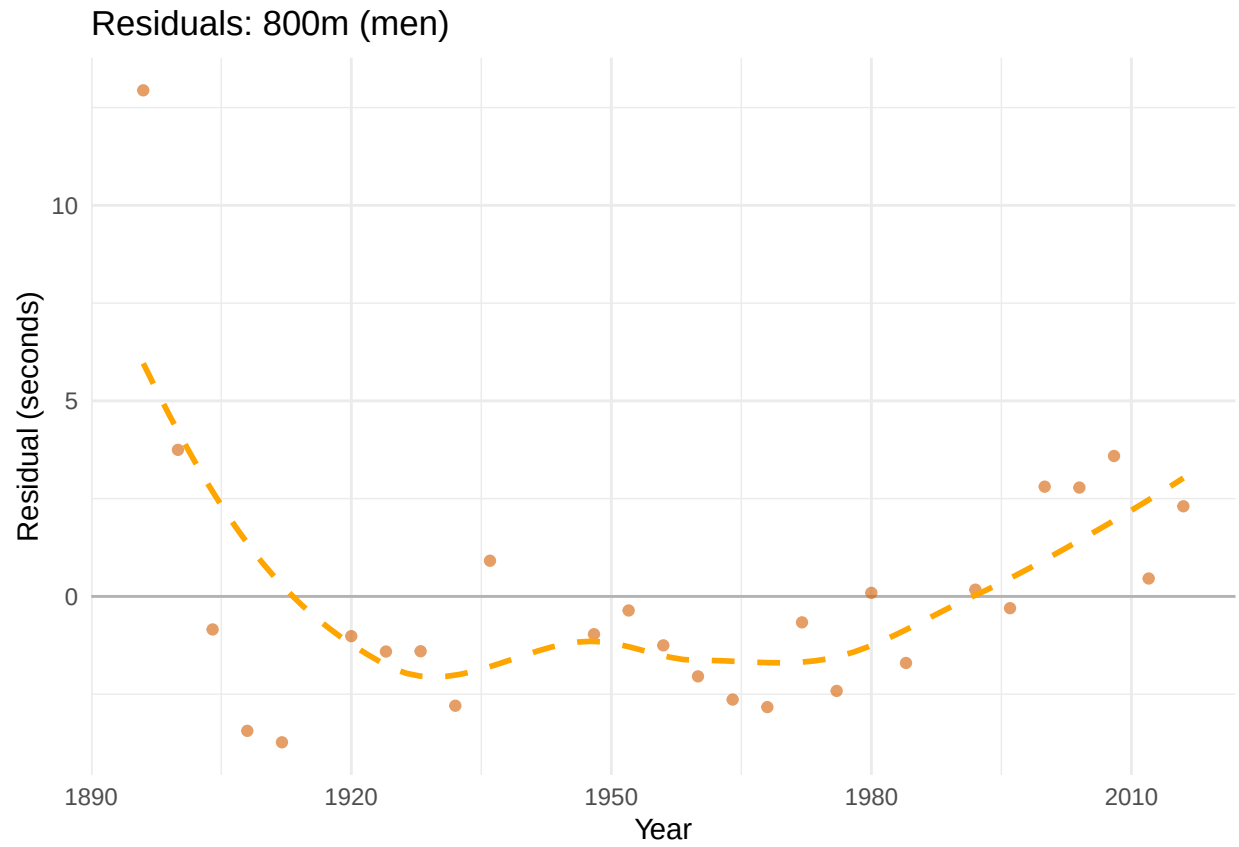


```
## 'geom_smooth()' using formula = 'y ~ x'
```

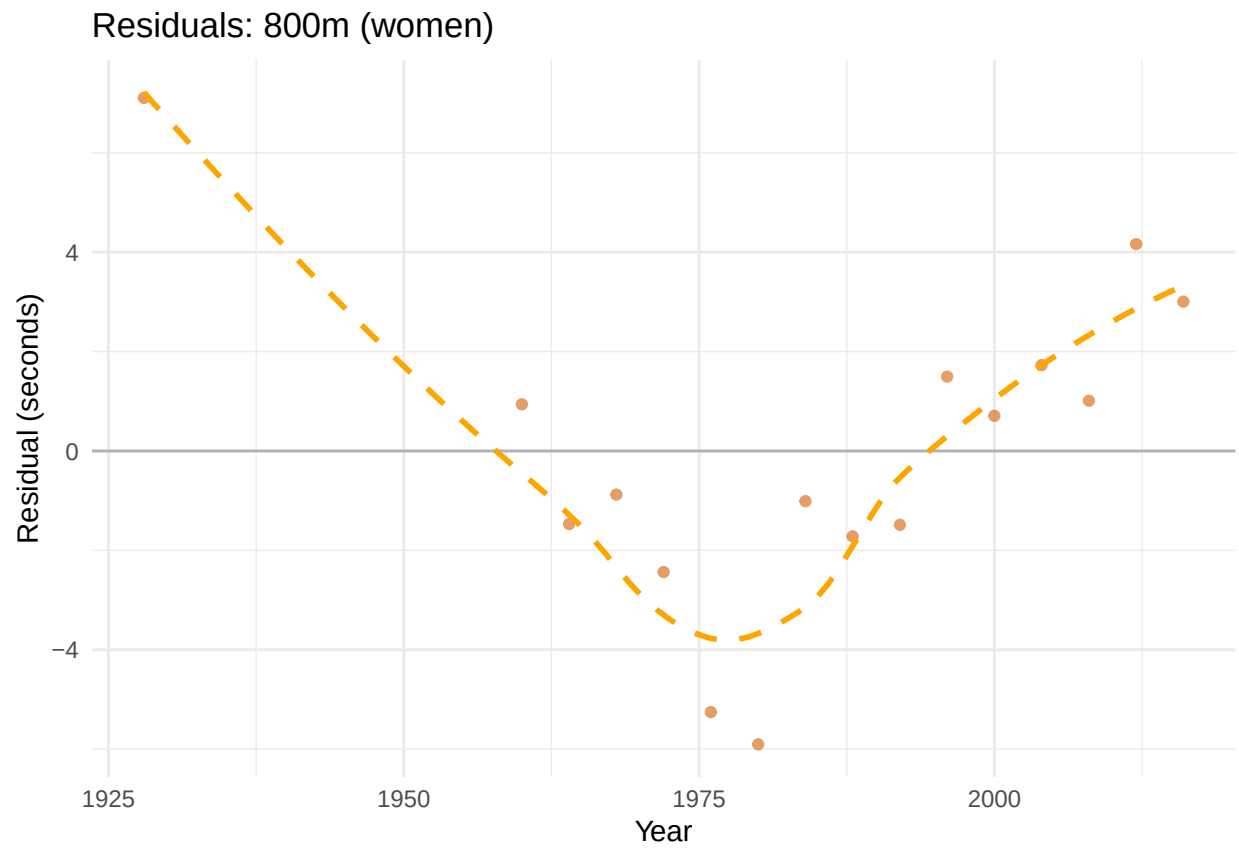
Residuals: 400m (women)



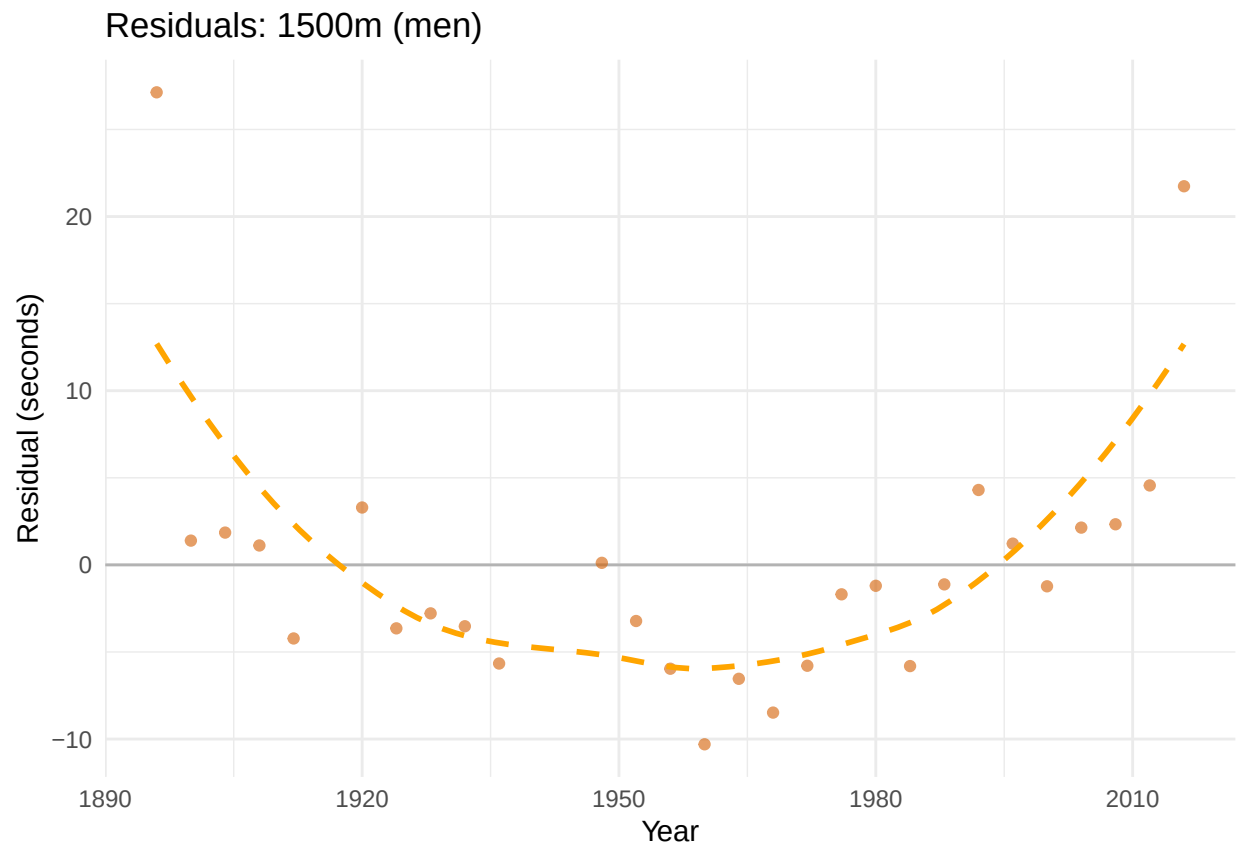
```
## 'geom_smooth()' using formula = 'y ~ x'
```

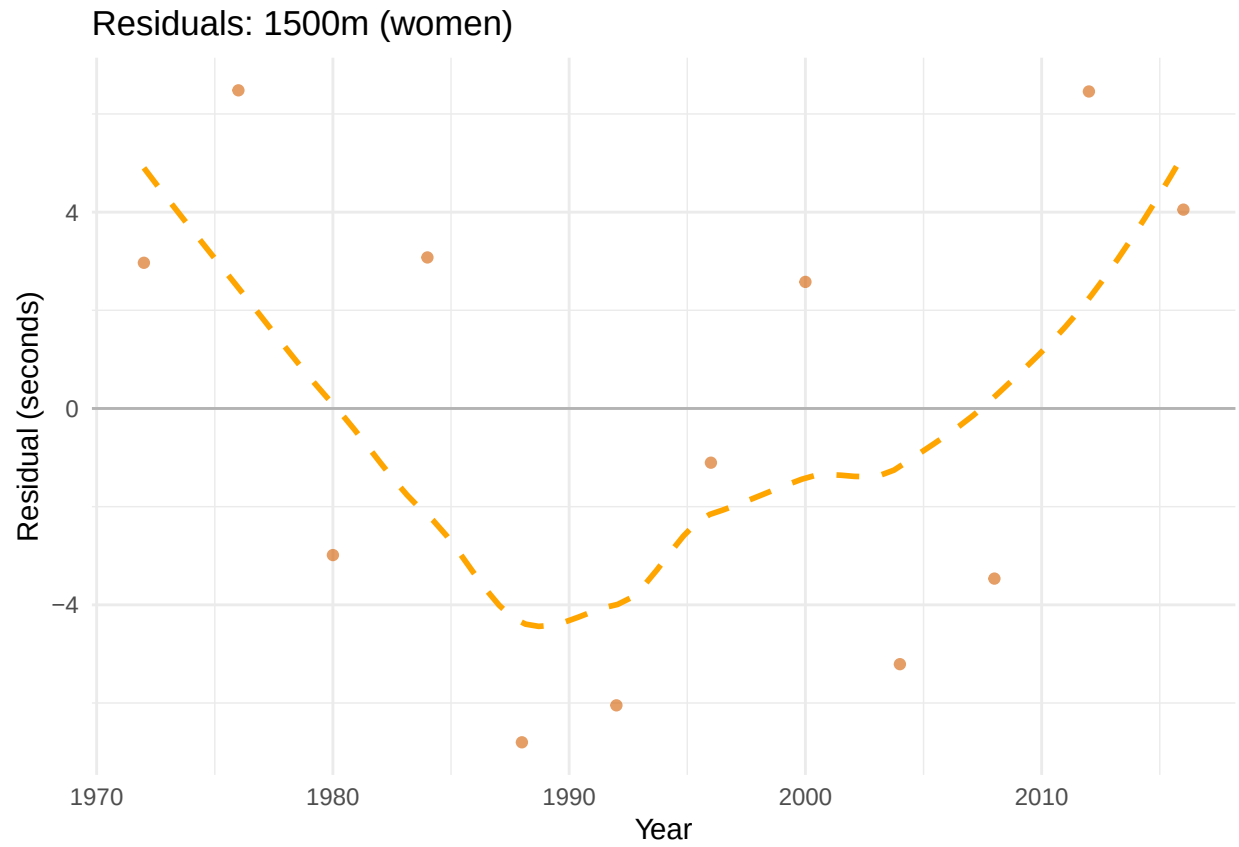
```
## 'geom_smooth()' using formula = 'y ~ x'
```



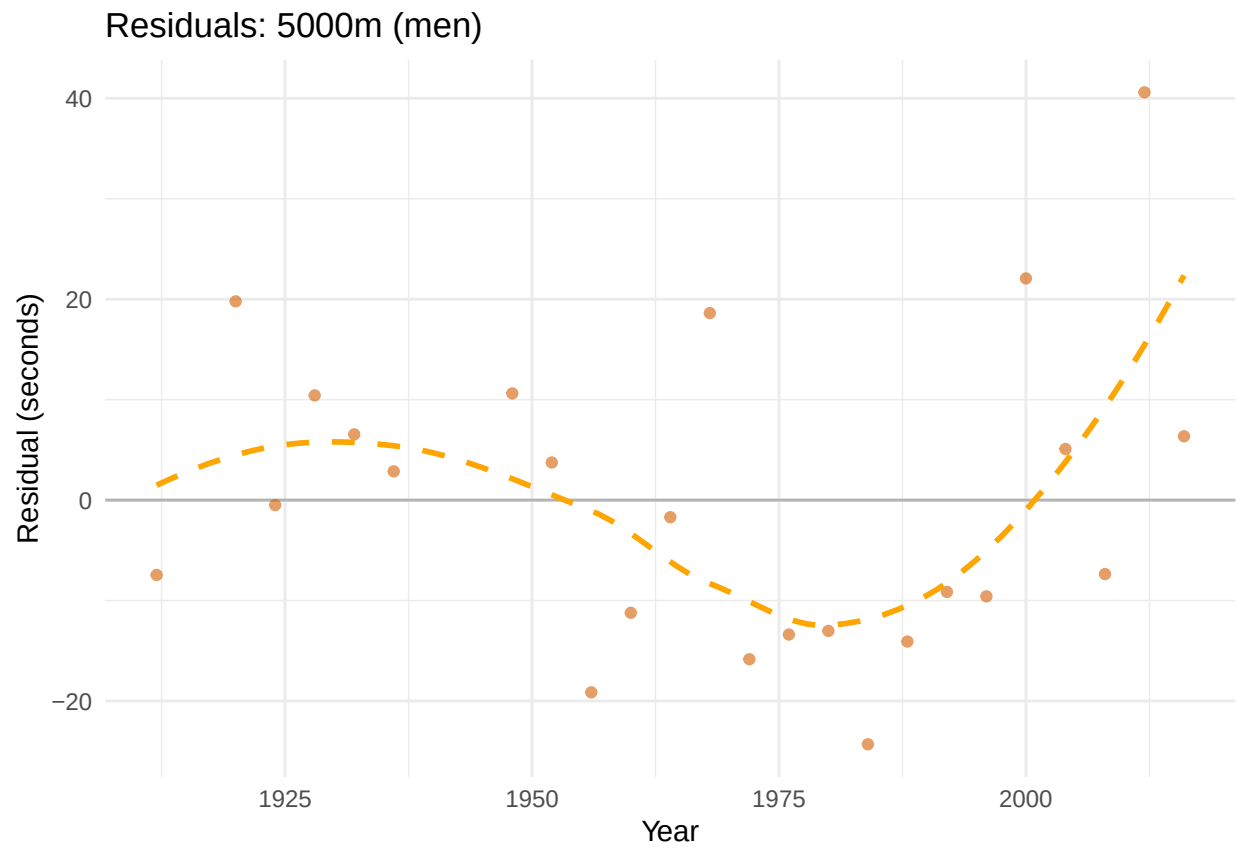
```
## 'geom_smooth()' using formula = 'y ~ x'
```



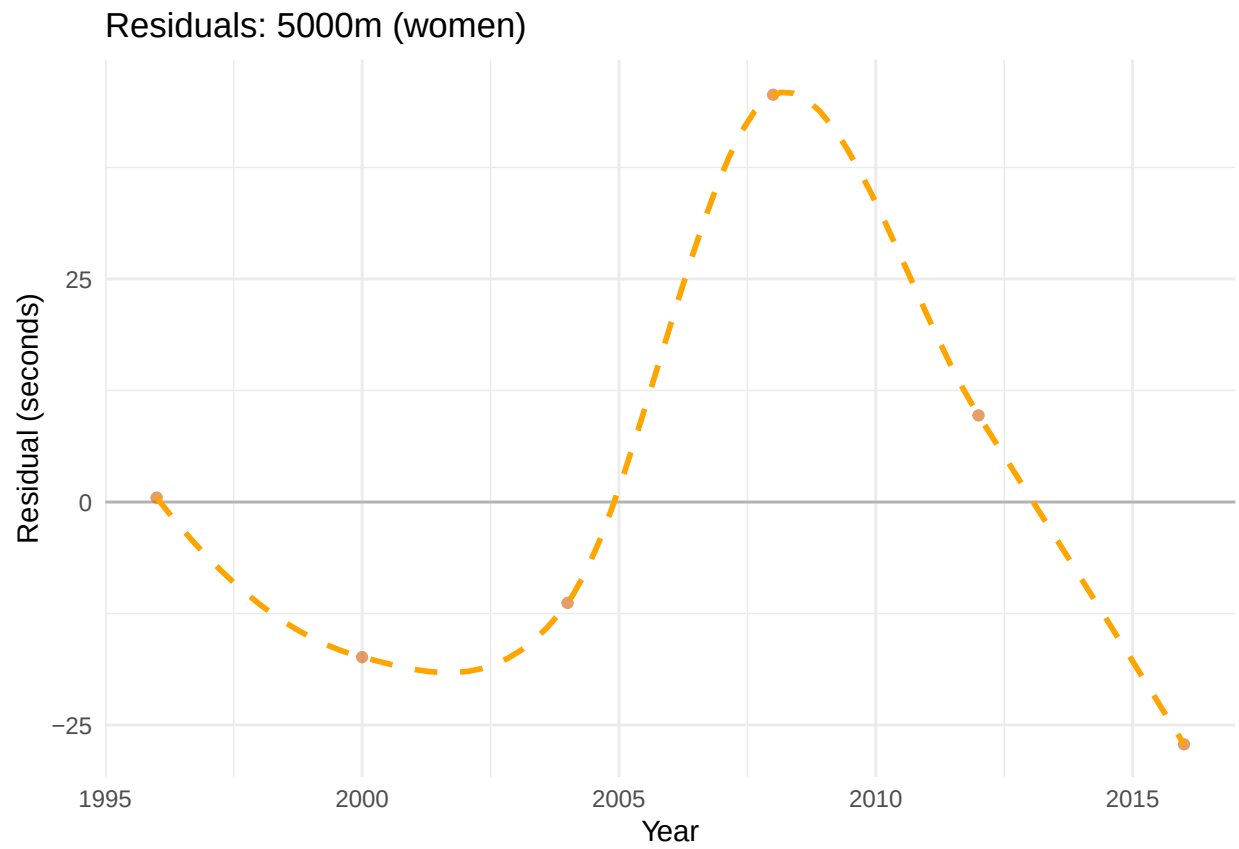
```
## 'geom_smooth()' using formula = 'y ~ x'
```



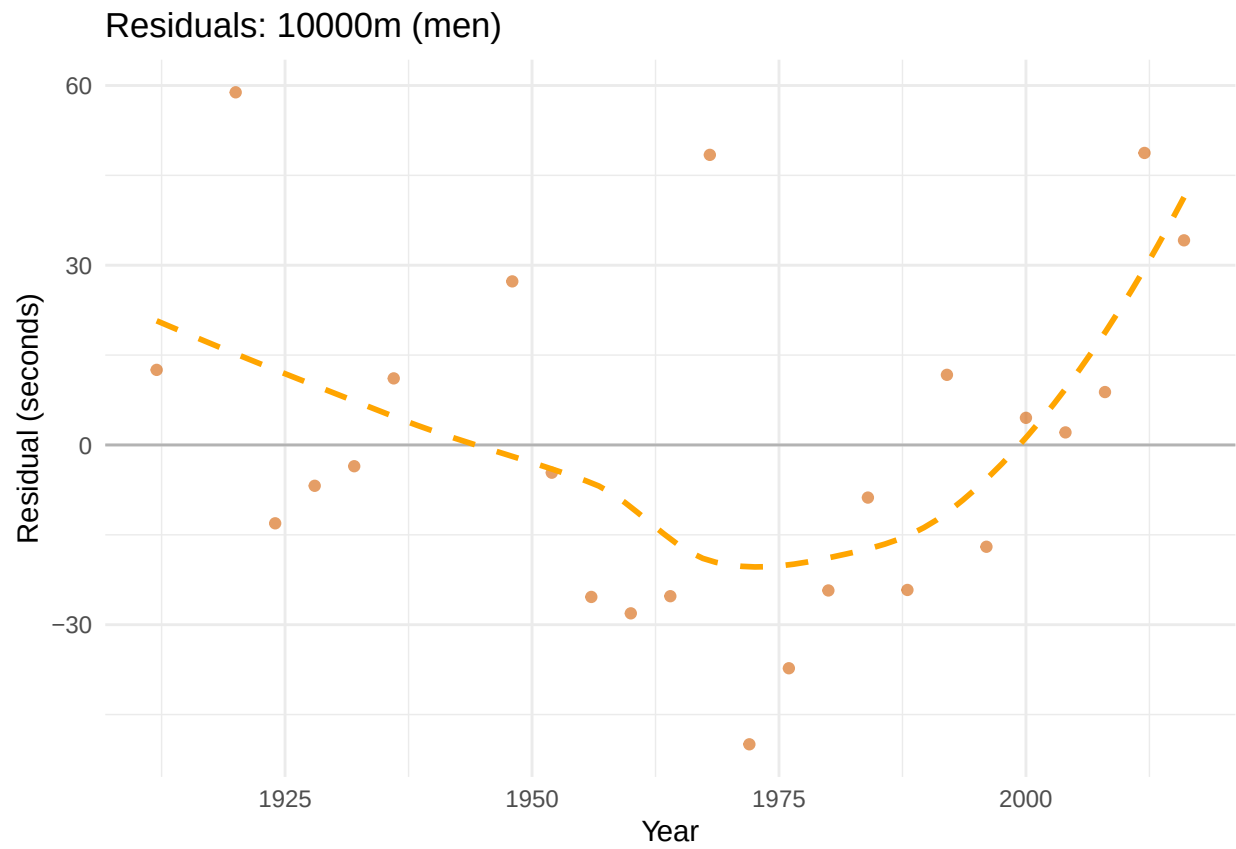
```
## 'geom_smooth()' using formula = 'y ~ x'
```



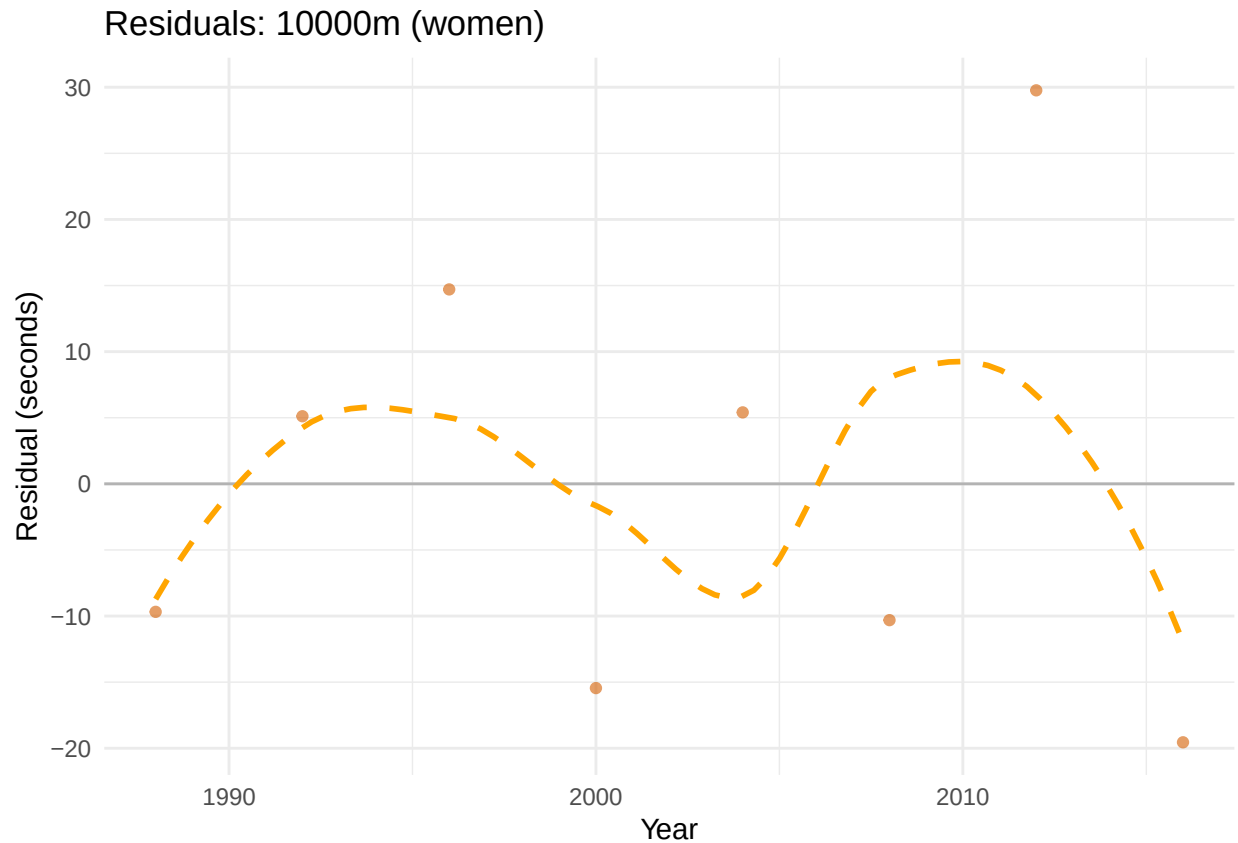
```
## 'geom_smooth()' using formula = 'y ~ x'
```



```
## 'geom_smooth()' using formula = 'y ~ x'
```



```
## 'geom_smooth()' using formula = 'y ~ x'
```



Interpretation of Residual Plots:

The residual plots reveal important insights about the limitations of the linear models:

1. **Non-Random Patterns:** Many events show clear non-random patterns in the residuals, particularly curved or U-shaped trends. This indicates that the linear model does not fully capture the relationship between time and year.
2. **Systematic Bias:** The curved patterns suggest that linear models systematically underestimate winning times in the middle years and overestimate them in early and recent years for many events.
3. **Heteroscedasticity:** Some events show changing variance in residuals over time, violating the assumption of constant variance.
4. **Non-Linear Trends:** The clear patterns indicate that improvements in Olympic performance have not been strictly linear. Most events show evidence of early rapid improvement followed by slowing gains in recent decades.
5. **Model Limitations:** These patterns suggest that a non-linear model (such as quadratic or piecewise regression) might be more appropriate for modeling these trends, particularly for capturing the apparent slowing of improvement rates in recent decades.

Part 1.4 Predict the winning time for each race in the 2020 and 2024 Olympics. Give a prediction interval for your forecasts. What assumptions have you made in these calculations?

```
# Create future years to forecast
future_years <- tibble(Year = c(2020, 2024))

# Generate predictions from existing models
forecast_table <- regression_results %>%
  mutate(
    predictions = map(model, ~ predict(.x, newdata = future_years, interval = "prediction") %>%
      as_tibble() %>%
      bind_cols(future_years))
  ) %>%
  unnest(predictions) %>%
  rename(Fit = fit, Lower = lwr, Upper = upr) %>%
  arrange(Sex, Year, Length)

# Format as display table
forecast_table %>%
  select(Length, Sex, Year, Fit, Lower, Upper) %>%
  mutate(across(c(Fit, Lower, Upper), ~round(.x, 2))) %>%
  gt() %>%
  tab_header(
    title = "Olympic Winning Time Forecasts for 2020 and 2024",
    subtitle = "Predicted values with 95% prediction intervals"
  ) %>%
  cols_label(
    Length = "Distance (m)",
    Sex = "Gender",
    Year = "Year",
    Fit = "Predicted Time",
    Lower = "Lower Bound",
    Upper = "Upper Bound"
  )
)
```

```
# Plot historical data with forecasts
for (i in 1:nrow(event_gender_combos)) {
  dist <- event_gender_combos$Length[i]
  gender <- event_gender_combos$Sex[i]

  # Historical data
  df_hist <- olympic_clean %>%
    filter(Length == dist, Sex == gender) %>%
    mutate(Type = "Observed") %>%
    select(Year, Time, Type)

  # Forecasted data
  df_forecast <- forecast_table %>%
    filter(Length == dist, Sex == gender) %>%
    mutate(Type = "Forecast") %>%
    select(Year, Time = Fit, Lower, Upper, Type)
}
```

Olympic Winning Time Forecasts for 2020 and 2024

Predicted values with 95% prediction intervals

Distance (m)	Gender	Year	Predicted Time	Lower Bound	Upper Bound
100	men	2020	9.53	9.03	10.04
200	men	2020	19.12	18.39	19.85
400	men	2020	42.04	39.55	44.53
800	men	2020	99.24	91.78	106.70
1500	men	2020	207.00	189.27	224.73
5000	men	2020	772.82	737.81	807.84
10000	men	2020	1580.35	1515.75	1644.95
100	men	2024	9.48	8.97	10.00
200	men	2024	19.02	18.28	19.75
400	men	2024	41.78	39.28	44.29
800	men	2024	98.63	91.12	106.14
1500	men	2024	205.74	187.90	223.57
5000	men	2024	768.71	733.40	804.01
10000	men	2024	1569.69	1504.56	1634.82
100	women	2020	10.53	10.02	11.04
200	women	2020	21.20	20.02	22.38
400	women	2020	48.44	45.87	51.01
800	women	2020	111.48	103.41	119.56
1500	women	2020	245.46	232.19	258.72
5000	women	2020	892.11	782.40	1001.83
10000	women	2020	1763.02	1706.69	1819.35
100	women	2024	10.48	9.96	10.99
200	women	2024	21.07	19.86	22.27
400	women	2024	48.28	45.64	50.91
800	women	2024	110.69	102.49	118.89
1500	women	2024	246.04	232.31	259.77
5000	women	2024	890.90	768.49	1013.32
10000	women	2024	1749.03	1688.67	1809.39

```

# Combine for plotting
df_combined <- bind_rows(
  df_hist %>% select(Year, Time, Type),
  df_forecast %>% select(Year, Time, Type)
)

p <- ggplot() +
  # Historical data
  geom_line(data = df_hist, aes(x = Year, y = Time), color = "blue", size = 0.8) +
  geom_point(data = df_hist, aes(x = Year, y = Time), color = "blue", size = 2) +
  # Forecast
  geom_line(data = df_forecast, aes(x = Year, y = Time), color = "orange", linetype = "dashed", size = 0.8) +
  geom_point(data = df_forecast, aes(x = Year, y = Time), color = "orange", size = 3) +

```

```

# Prediction interval
geom_ribbon(data = df_forecast, aes(x = Year, ymin = Lower, ymax = Upper),
          fill = "orange", alpha = 0.2) +

labs(
  title = paste0(dist, "m (", gender, ") Olympic Winning Time Forecast"),
  x = "Year",
  y = "Time (seconds)"
) +
theme_minimal()

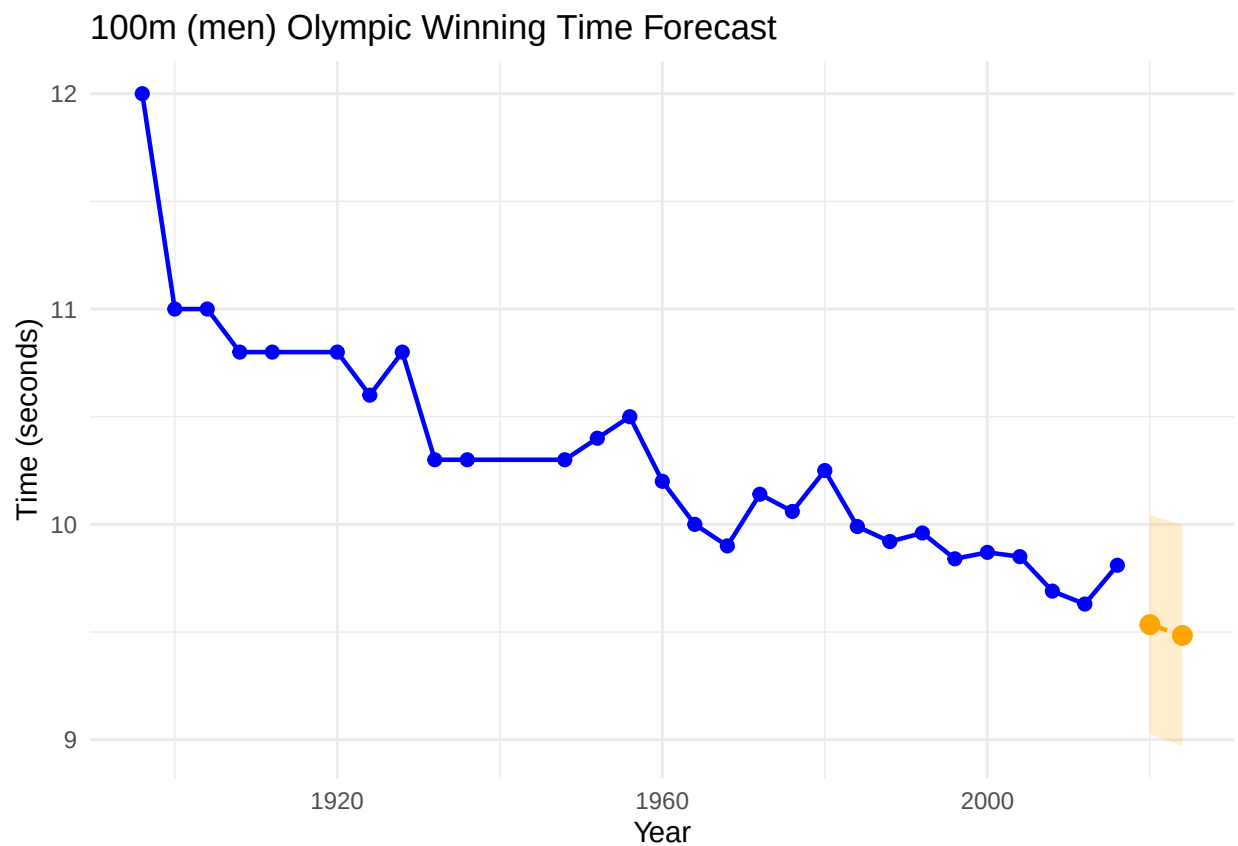
print(p)
}

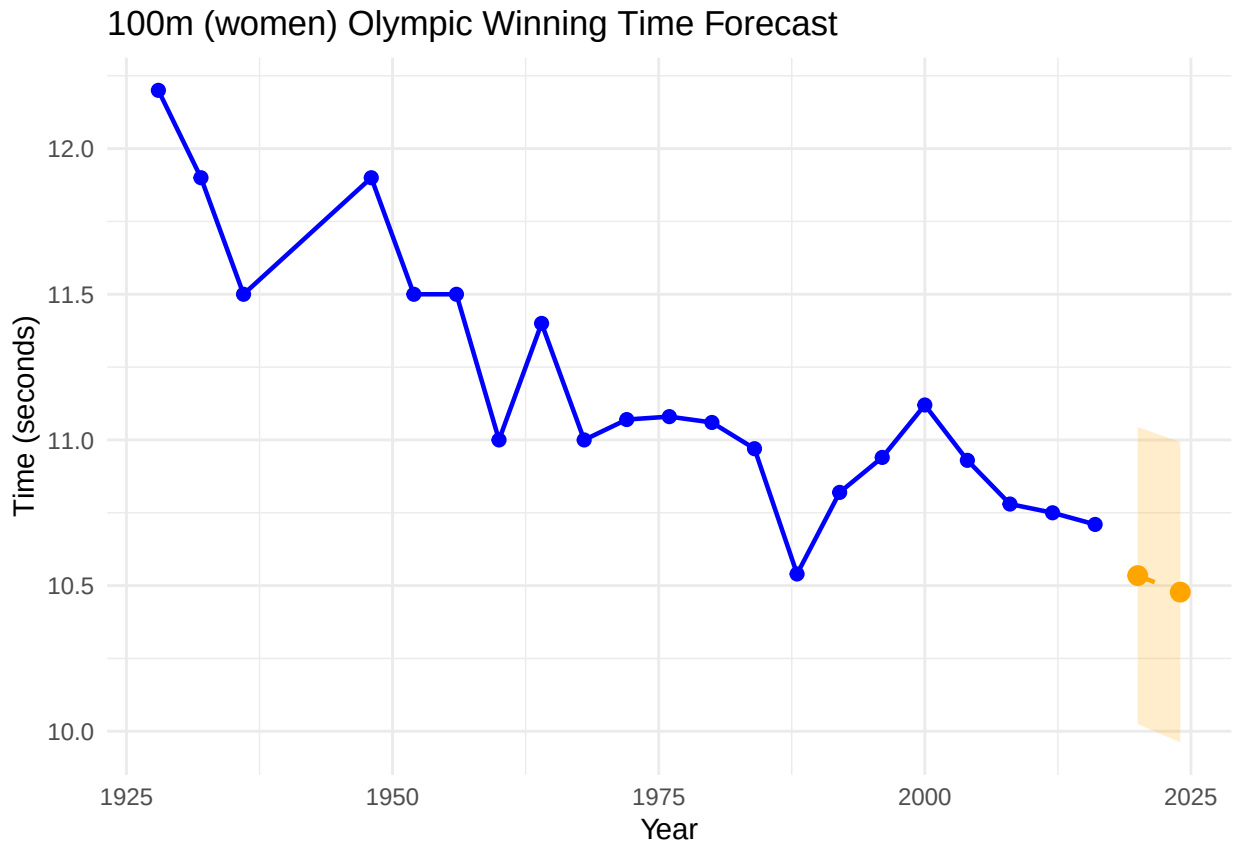
```

```

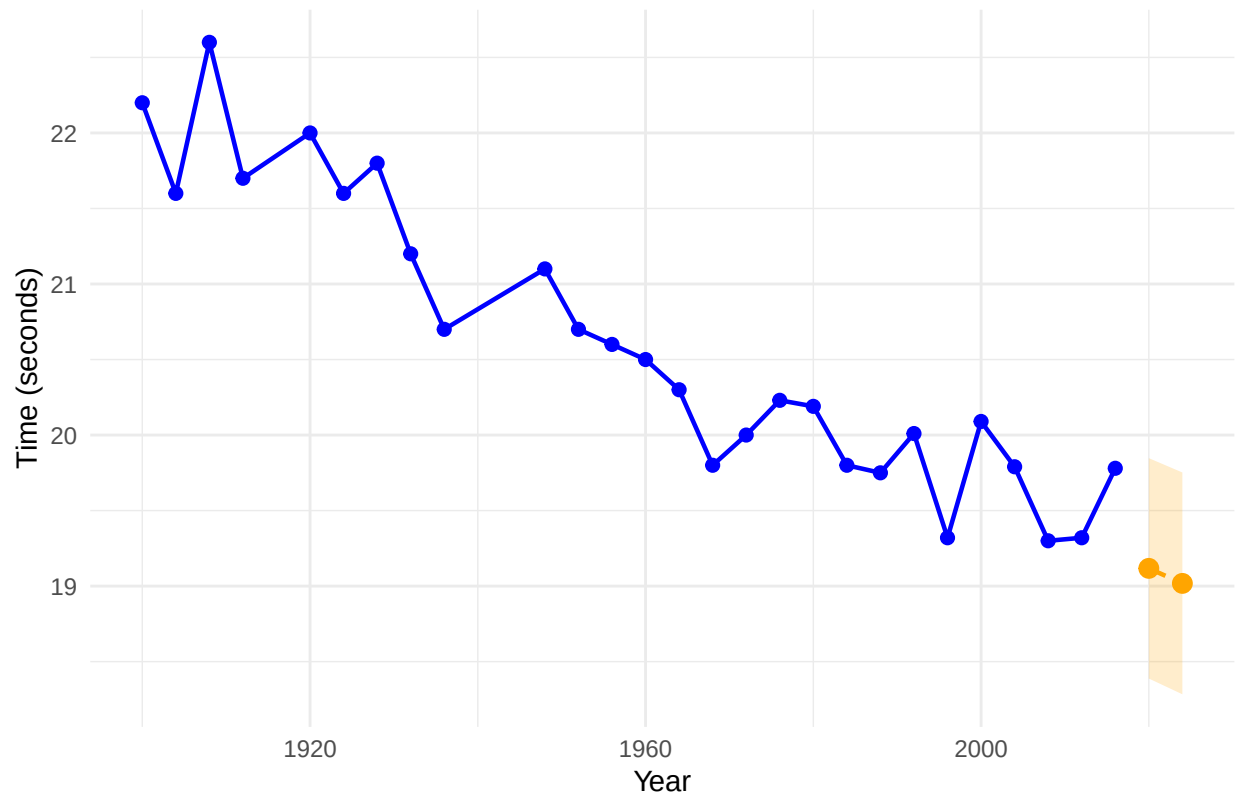
## Warning: Using 'size' aesthetic for lines was deprecated in ggplot2 3.4.0.
## i Please use 'linewidth' instead.
## This warning is displayed once every 8 hours.
## Call 'lifecycle::last_lifecycle_warnings()' to see where this warning was
## generated.

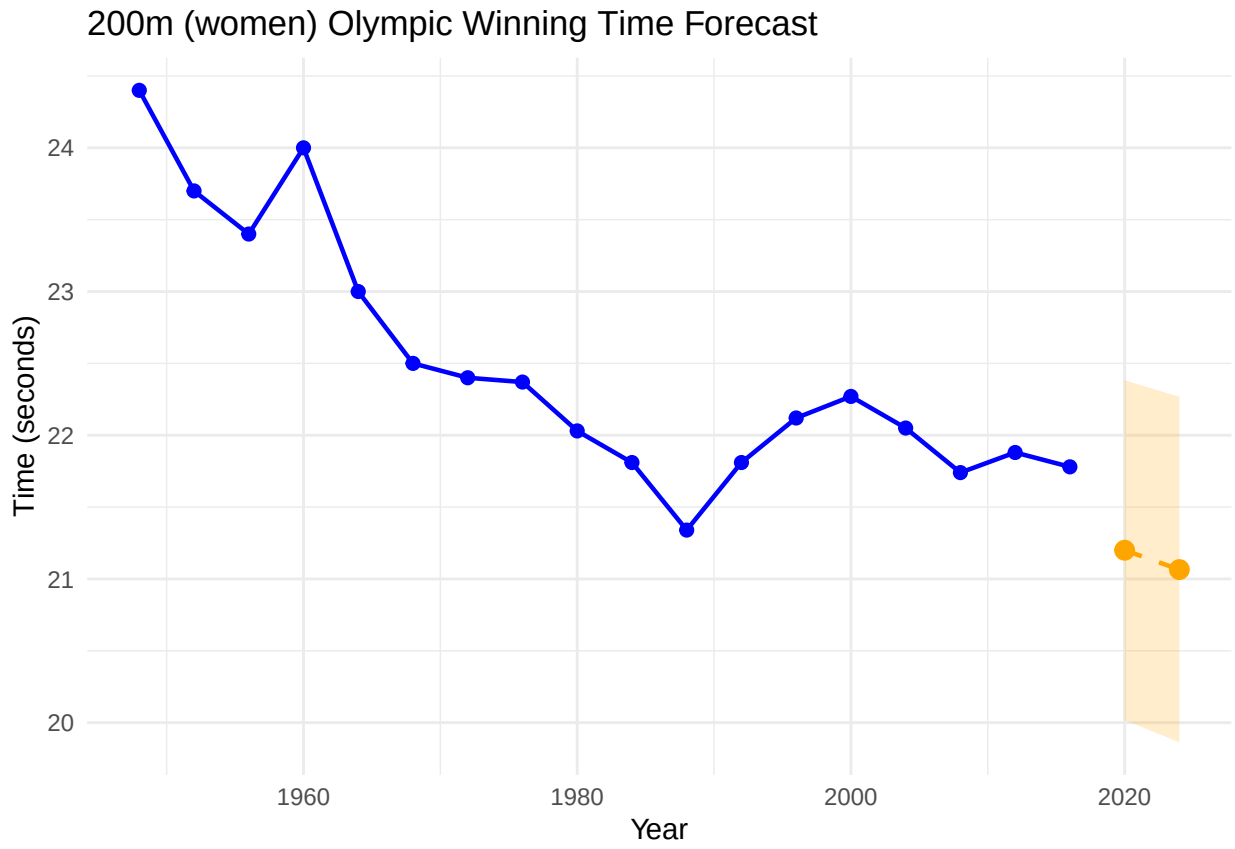
```

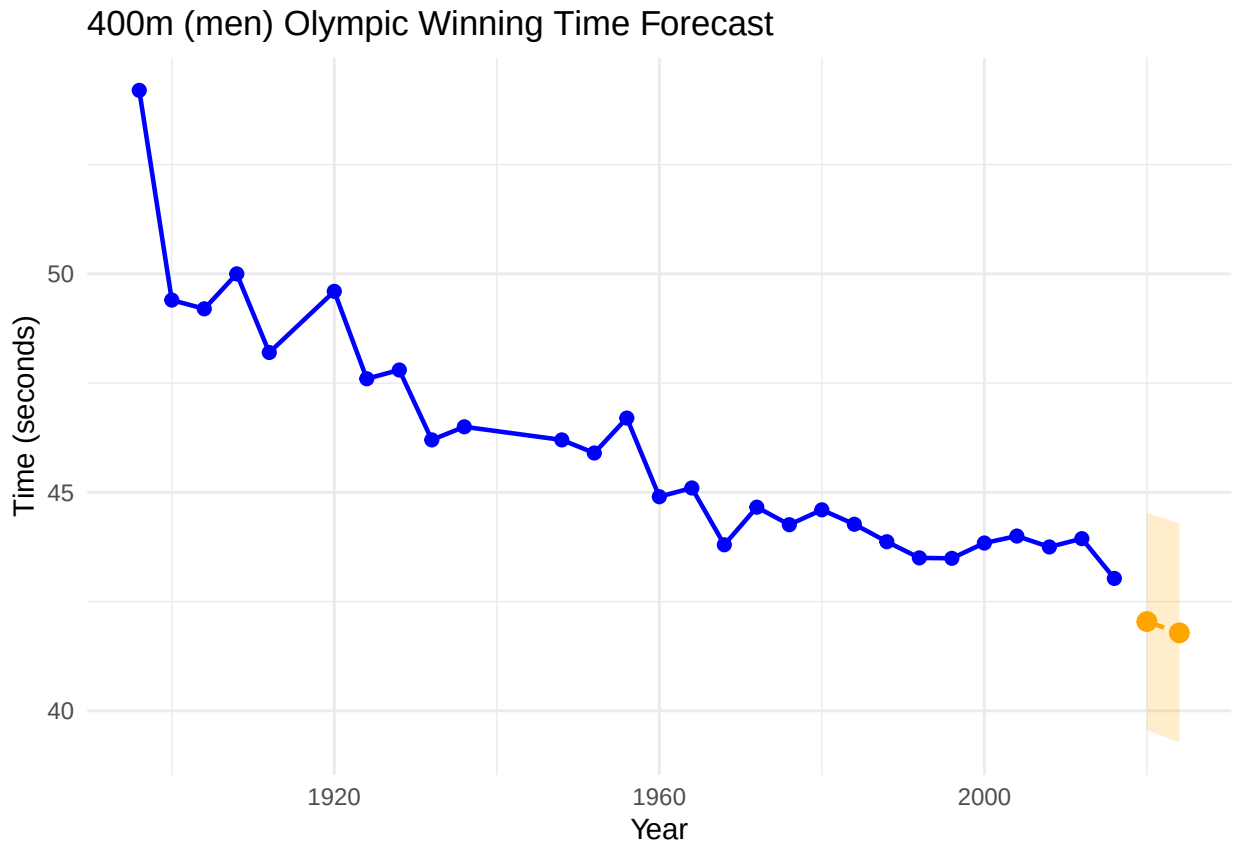


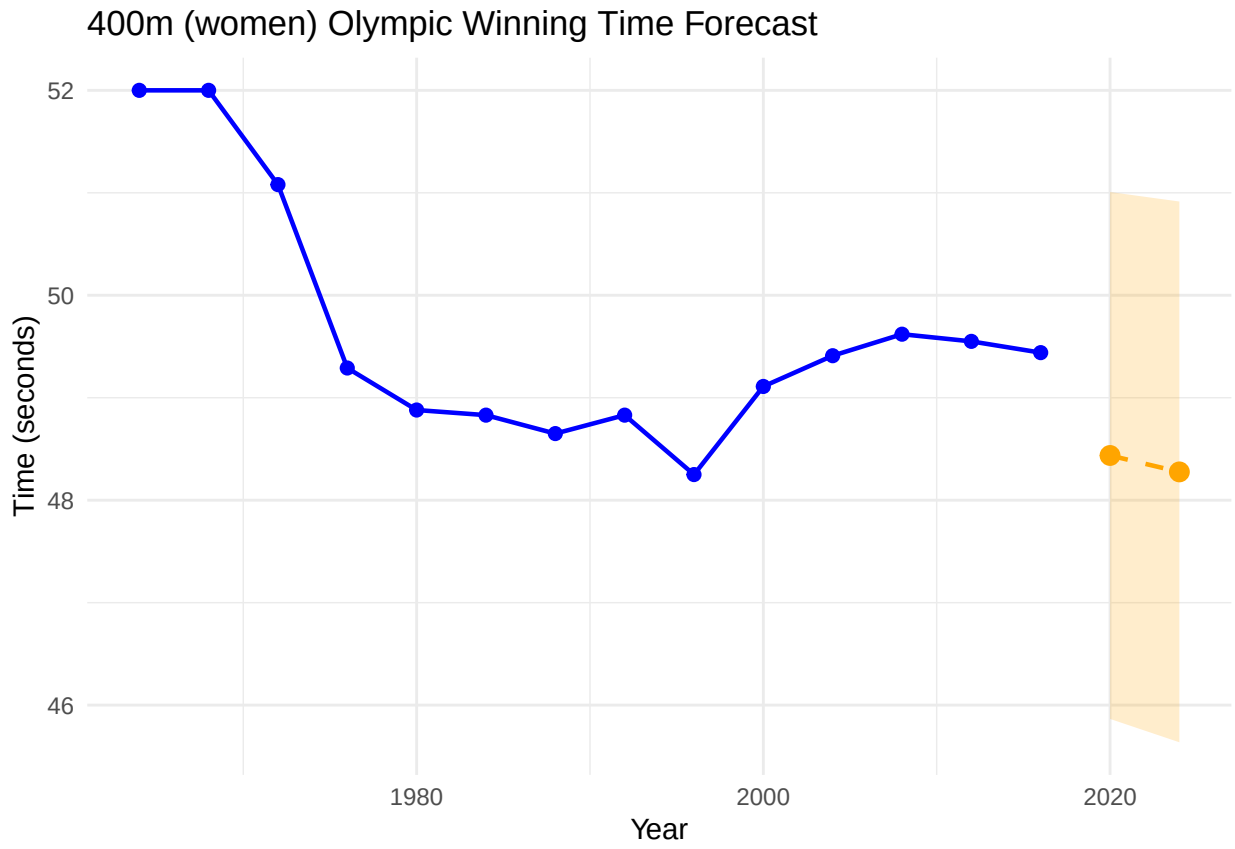


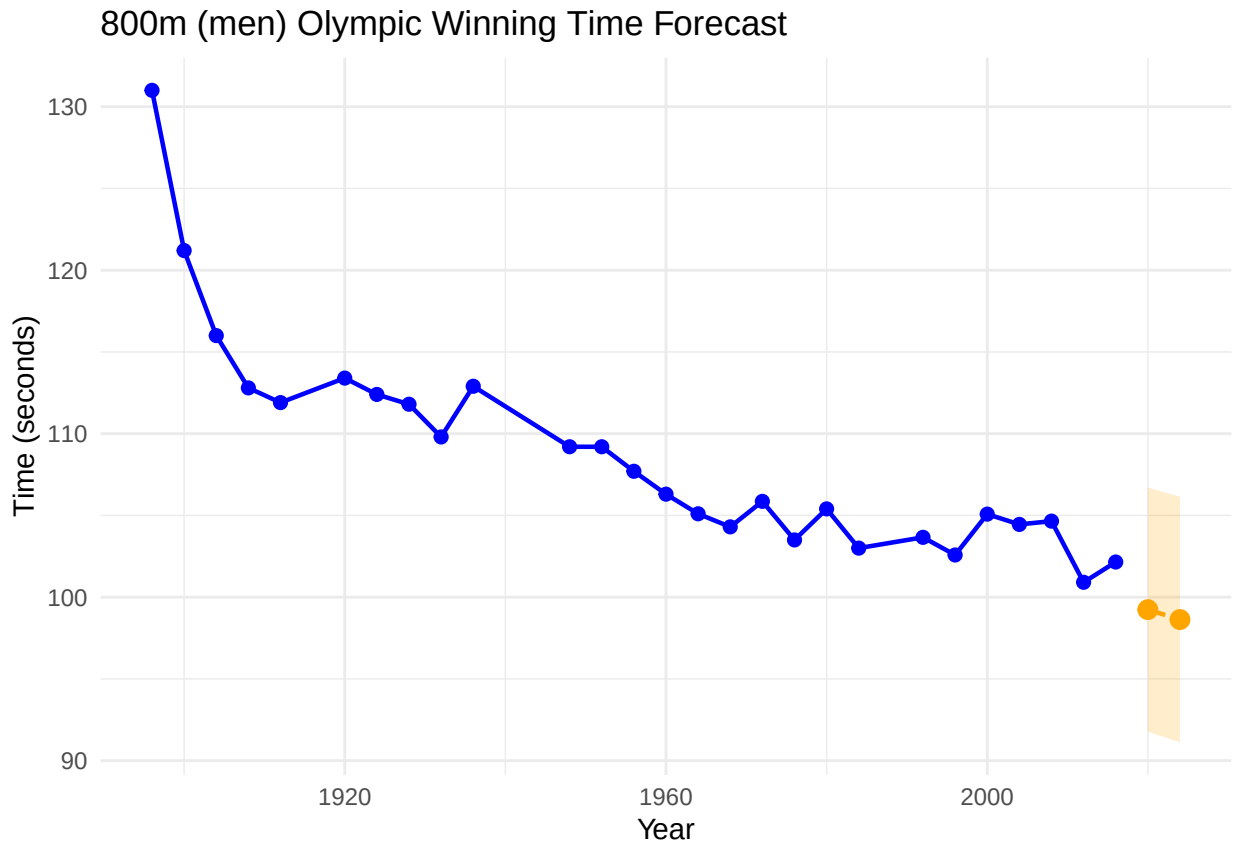
200m (men) Olympic Winning Time Forecast

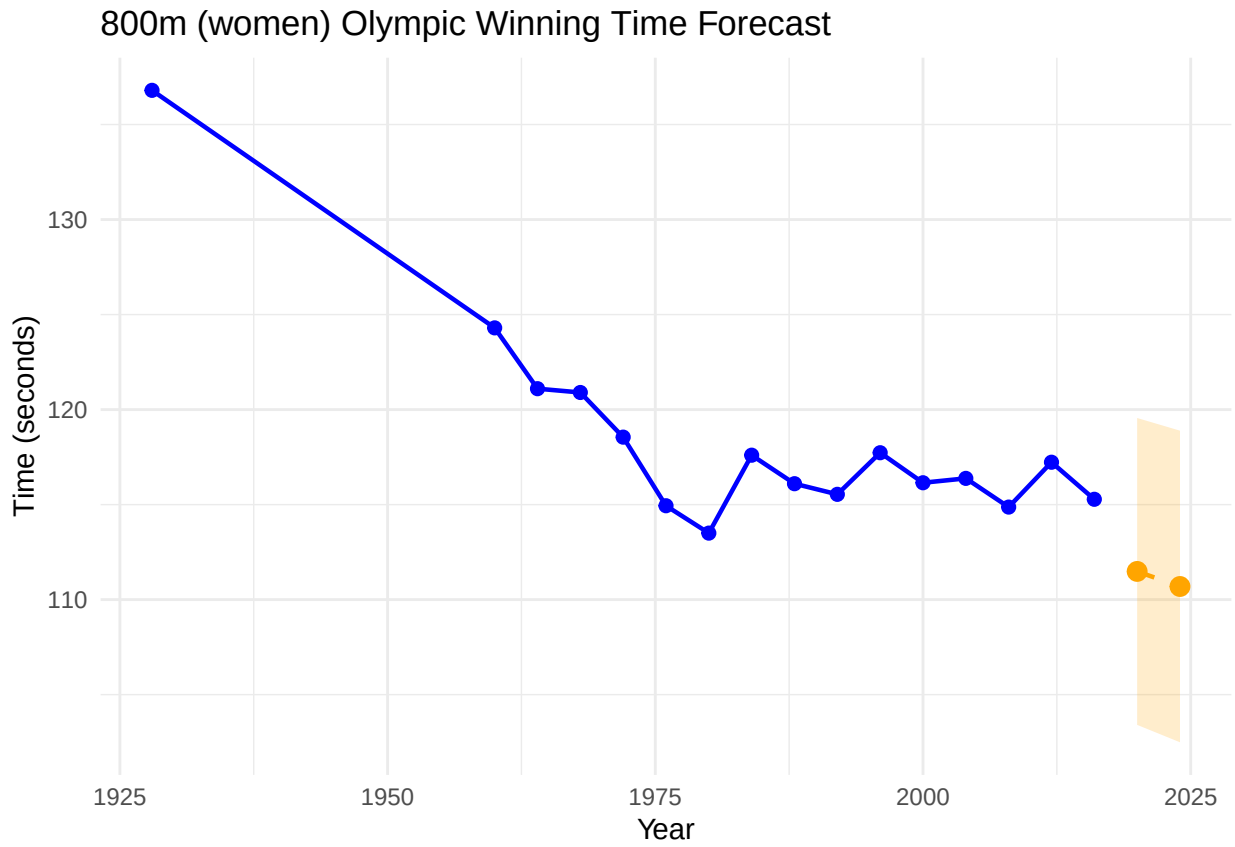


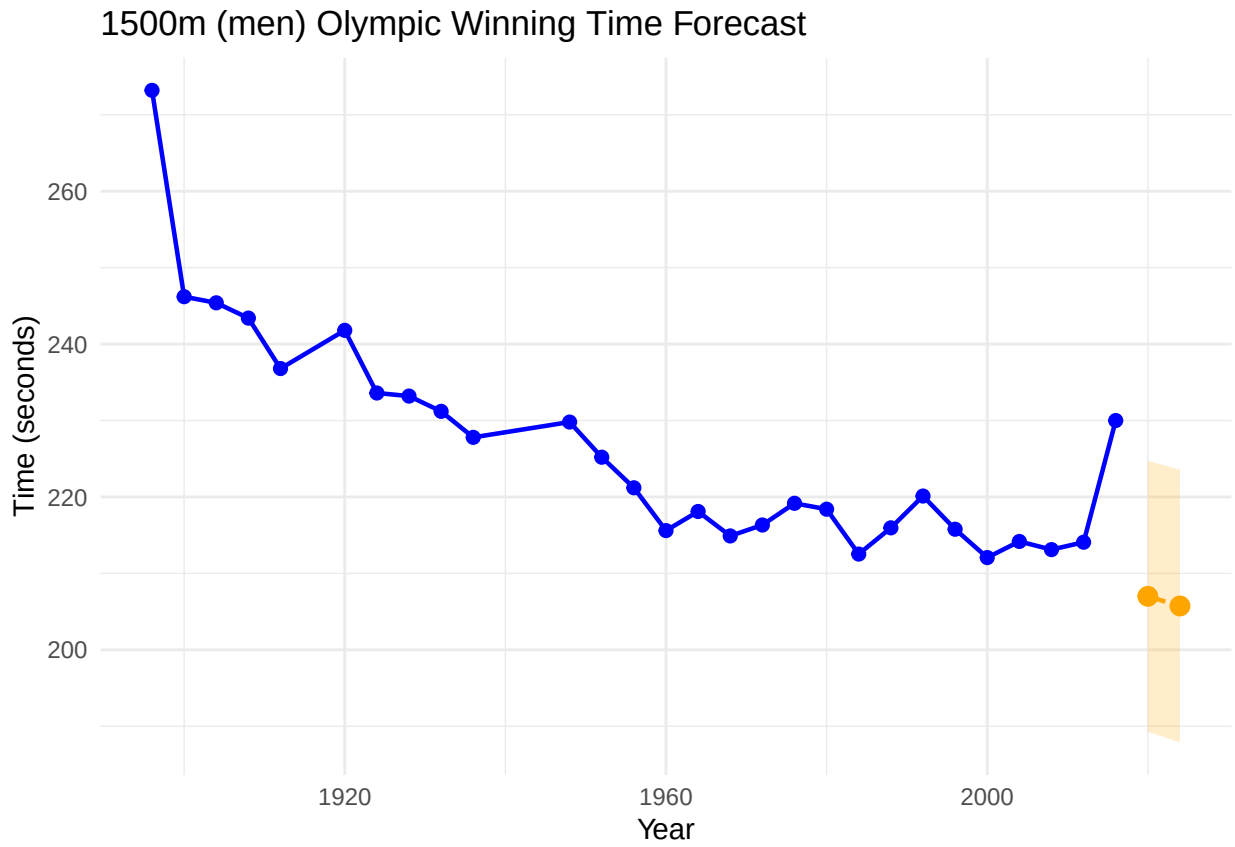


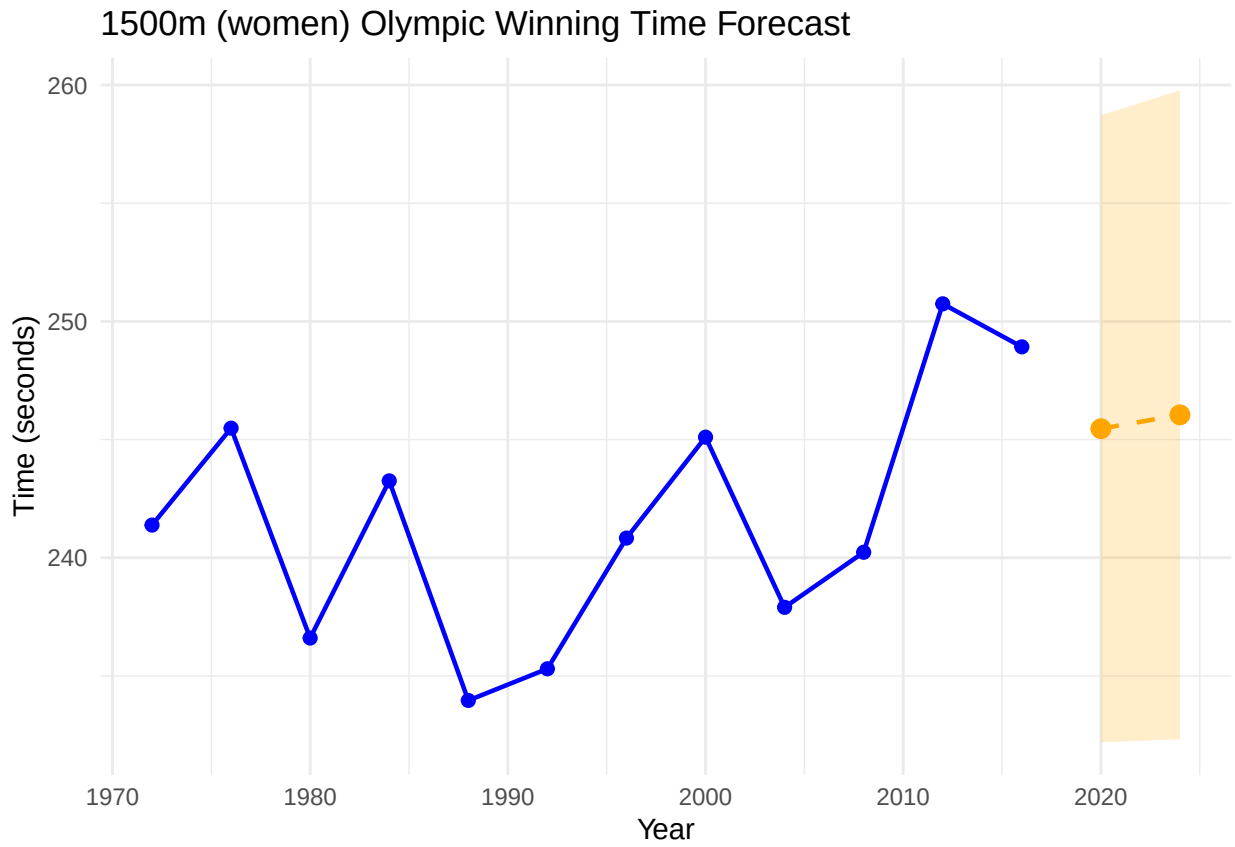


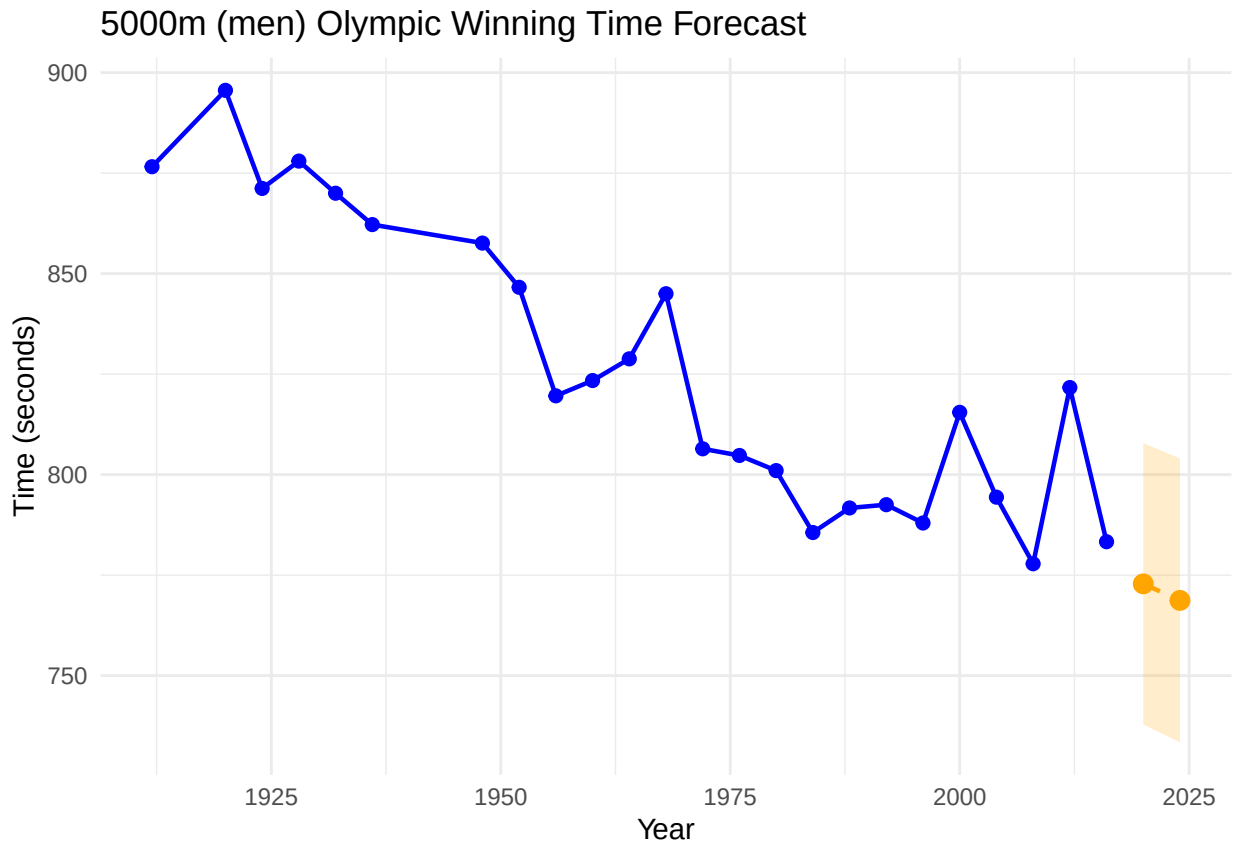




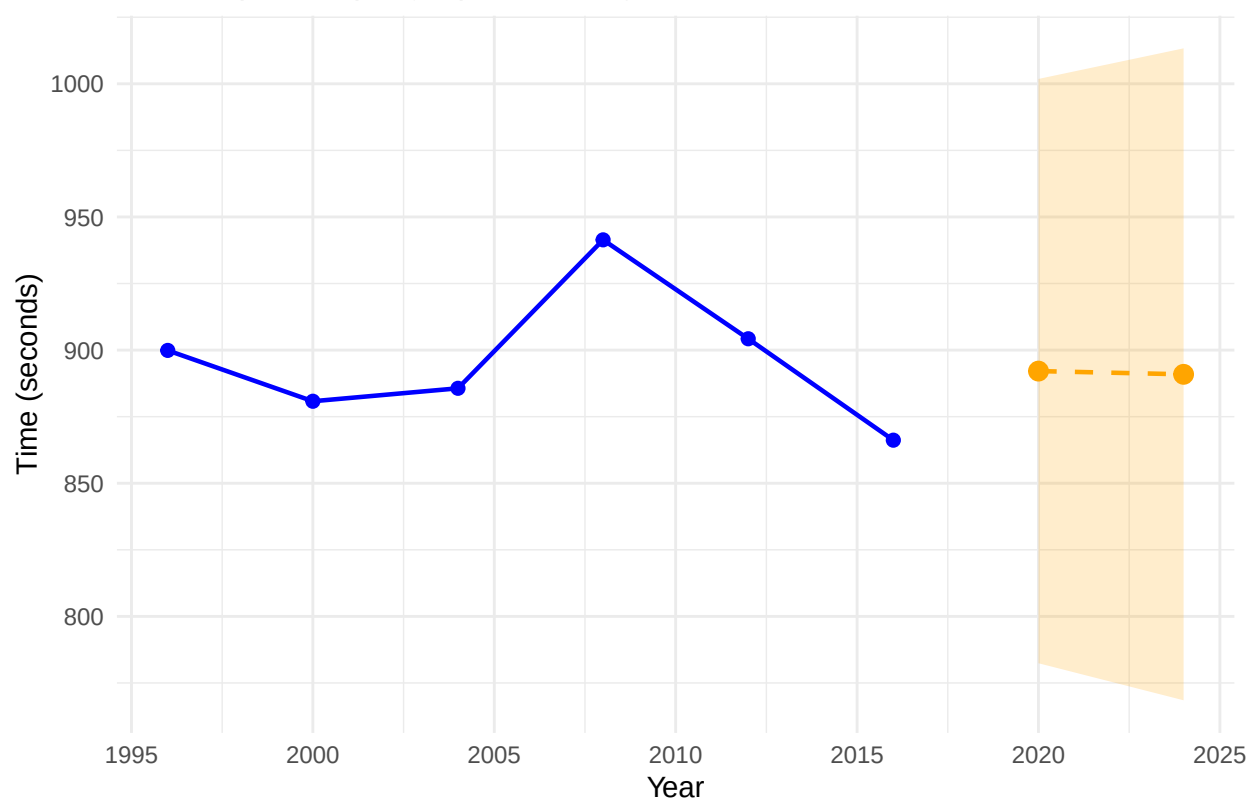


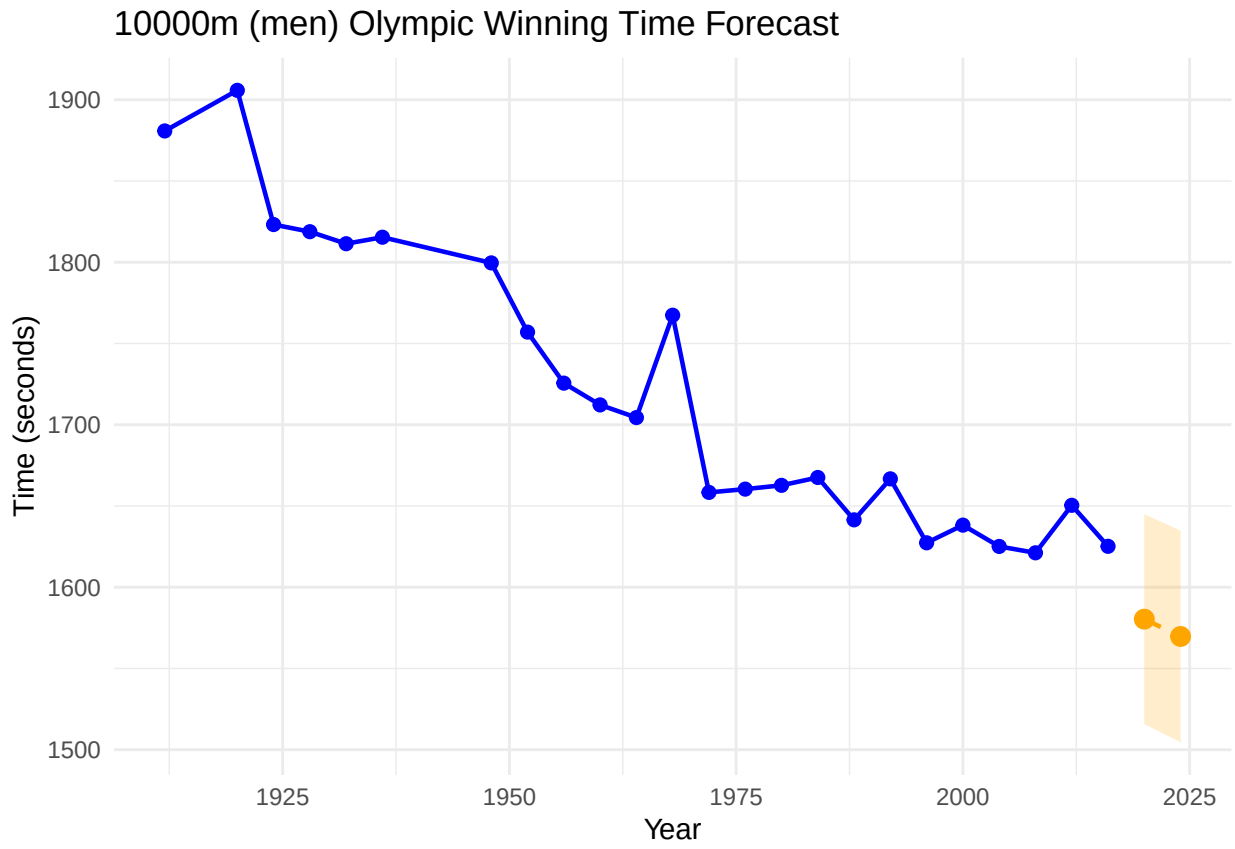


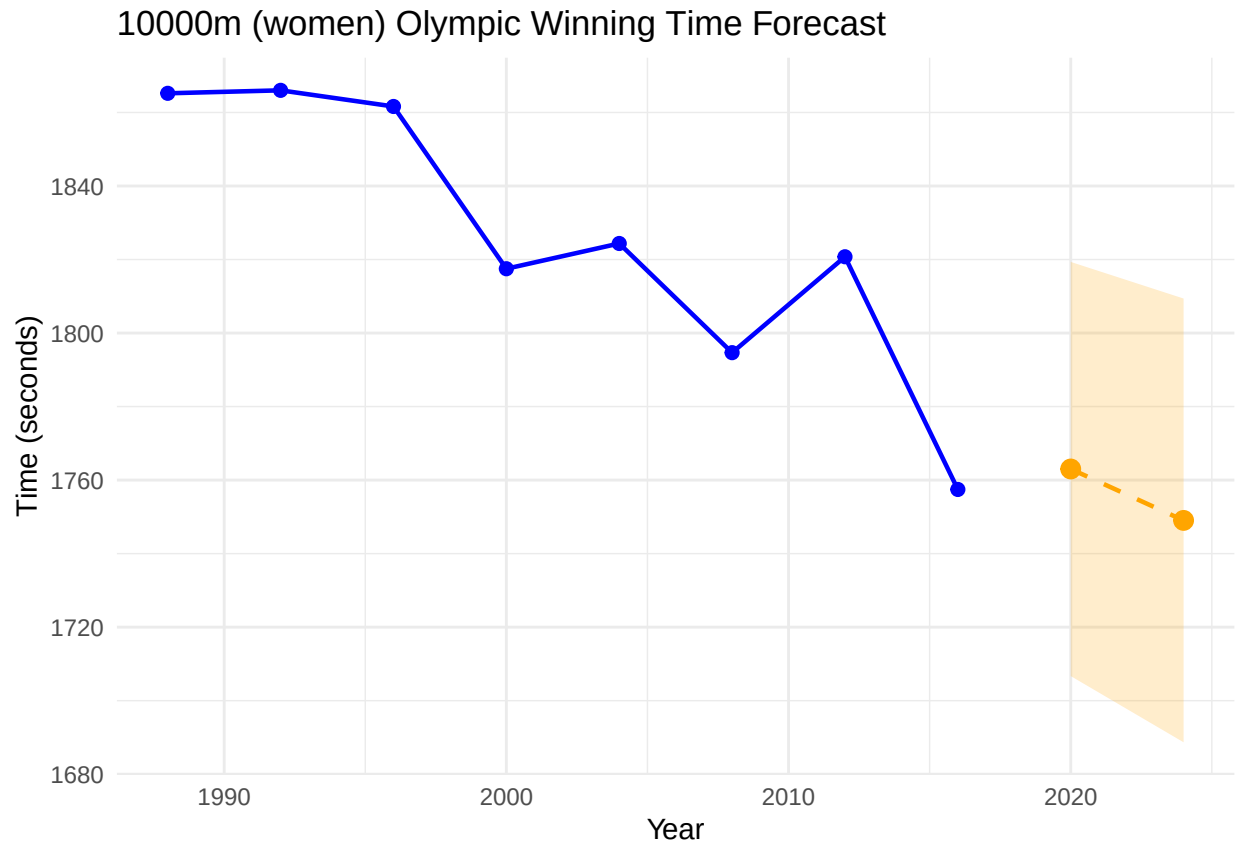




5000m (women) Olympic Winning Time Forecast







Assumptions in the Forecasting Model:

1. **Linearity:** We assume that the linear relationship between winning time and year continues to hold in the future. As seen in the residual analysis, this assumption may not be valid, especially given the apparent slowing of improvements in recent decades.
2. **Independence:** We assume that the residuals are independent over time, though there might be some temporal correlation in the data.
3. **Homoscedasticity:** We assume constant variance in the residuals across time, which may not hold as shown in some residual plots.
4. **Normality:** The prediction intervals assume normally distributed residuals.
5. **Continuity:** We assume that Olympic games would be held as scheduled, without considering potential disruptions (such as the actual postponement of the 2020 Olympics due to COVID-19).

Question 3: US Gasoline Series

The us gasoline series consists of weekly data for supplies of US finished motor gasoline product, from 2 February 1991 to 20 January 2017. The units are in “million barrels per day”. Consider only the data to the end of 2004.

3.1 Fit a harmonic regression with trend to the data. Experiment with changing the number of Fourier terms.

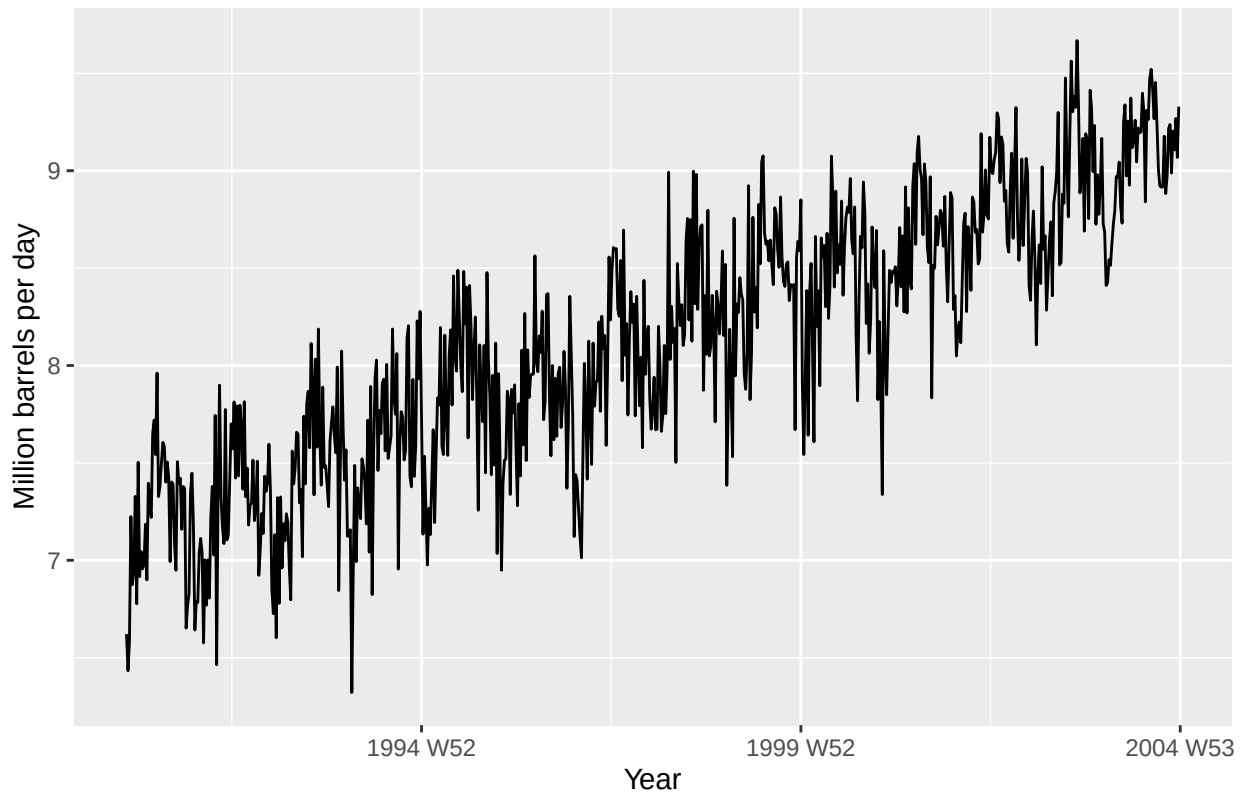
```
library(fpp3)
library(tidyverse)

# Load the dataset
data("us_gasoline")

# Filter data through the end of 2004
gasoline_2004 <- us_gasoline %>%
  filter(Week <= yearweek("2004 W52"))

# View the data structure
gasoline_2004 %>%
  autoplot(Barrels) +
  labs(title = "US Gasoline Supply (1991-2004)",
       y = "Million barrels per day",
       x = "Year")
```

US Gasoline Supply (1991–2004)



```
# Fit models with different K values
harmonic_model_k1 <- gasoline_2004 %>%
  model(TSLM(Barrels ~ trend() + fourier(period = 52, K = 1)))

harmonic_model_k2 <- gasoline_2004 %>%
```

```

model(TSLM(Barrels ~ trend() + fourier(period = 52, K = 2)))

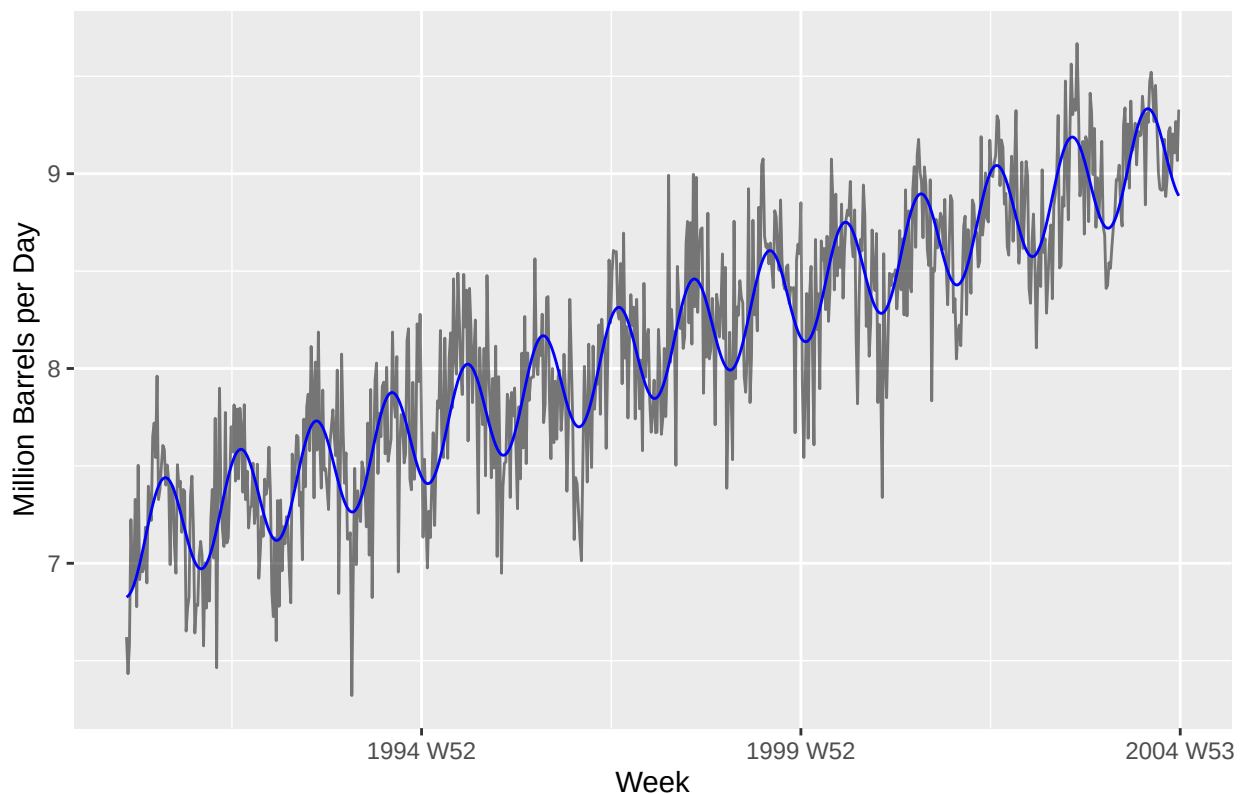
harmonic_model_k3 <- gasoline_2004 %>%
  model(TSLM(Barrels ~ trend() + fourier(period = 52, K = 3)))

harmonic_model_k4 <- gasoline_2004 %>%
  model(TSLM(Barrels ~ trend() + fourier(period = 52, K = 4)))

# Plot fitted values for different K values
harmonic_model_k1 %>%
  augment() %>%
  ggplot(aes(x = Week)) +
  geom_line(aes(y = Barrels), color = "black", alpha = 0.5) +
  geom_line(aes(y = .fitted), color = "blue") +
  labs(title = "Harmonic Regression (K=1)",
       y = "Million Barrels per Day",
       x = "Week")

```

Harmonic Regression (K=1)



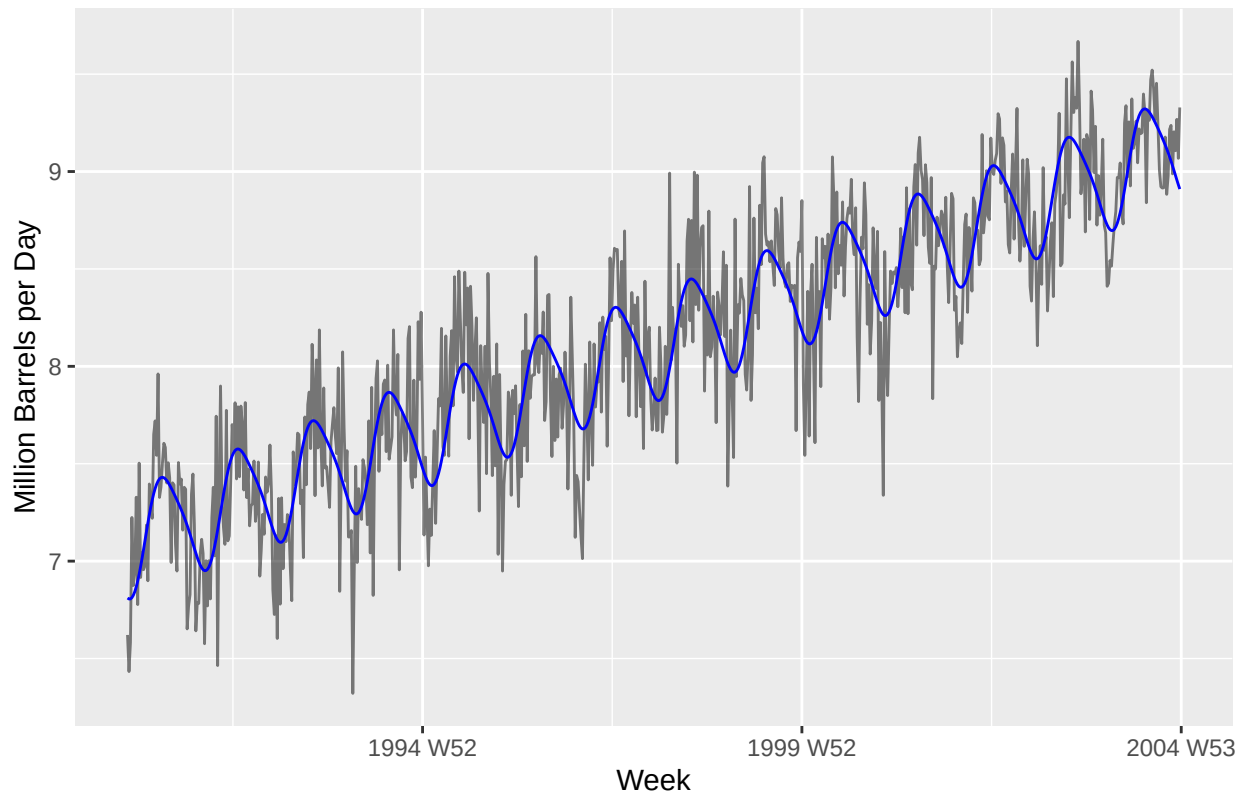
```

harmonic_model_k2 %>%
  augment() %>%
  ggplot(aes(x = Week)) +
  geom_line(aes(y = Barrels), color = "black", alpha=0.5) +
  geom_line(aes(y = .fitted), color = "blue") +
  labs(title = "Harmonic Regression (K=2)",
       y = "Million Barrels per Day",

```

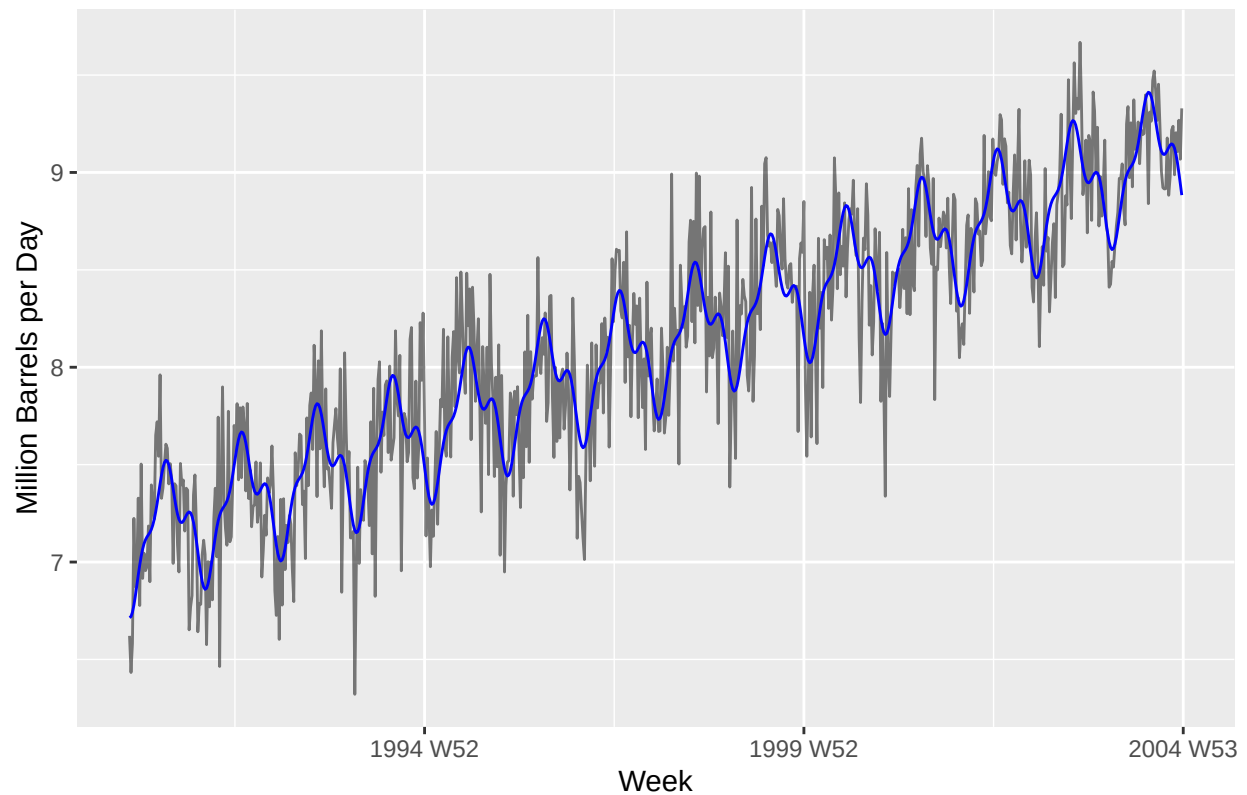
```
x = "Week")
```

Harmonic Regression (K=2)



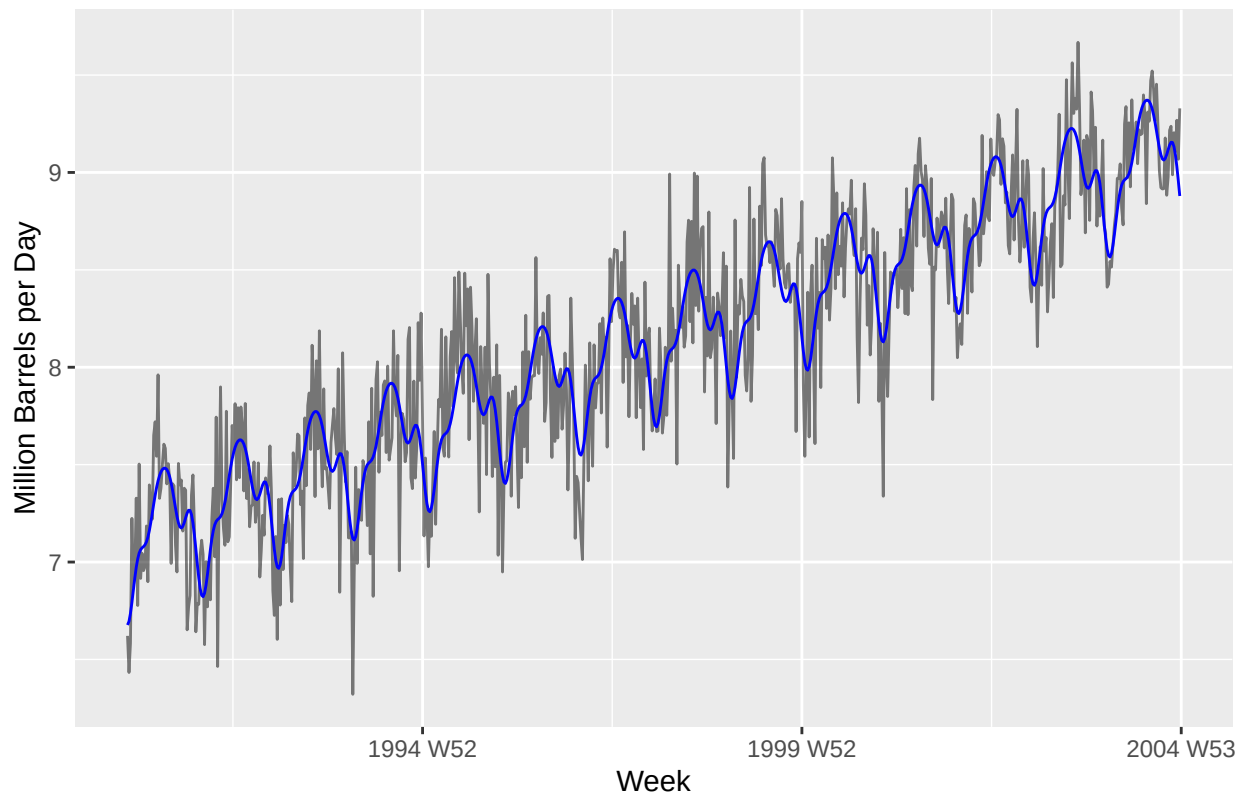
```
harmonic_model_k3 %>%  
  augment() %>%  
  ggplot(aes(x = Week)) +  
  geom_line(aes(y = Barrels), color = "black", alpha = 0.5) +  
  geom_line(aes(y = .fitted), color = "blue") +  
  labs(title = "Harmonic Regression (K=3)",  
        y = "Million Barrels per Day",  
        x = "Week")
```

Harmonic Regression (K=3)



```
harmonic_model_k4 %>%  
  augment() %>%  
  ggplot(aes(x = Week)) +  
  geom_line(aes(y = Barrels), color = "black", alpha = 0.5) +  
  geom_line(aes(y = .fitted), color = "blue") +  
  labs(title = "Harmonic Regression (K=4)",  
        y = "Million Barrels per Day",  
        x = "Week")
```

Harmonic Regression (K=4)



Observations on the models:

Looking at the fitted models with different K values:

- With K=1, the model captures the basic yearly pattern but misses many fluctuations.
- With K=2, the model improves and captures more seasonal variations.
- With K=3, the model further refines the fit, capturing more complex seasonal patterns.
- With K=4, there are not much in improvement, with only minor refinements over K=3.

The US gasoline supply shows clear seasonal patterns within each year, with peaks typically occurring during summer months when driving increases. There's also an overall upward trend over the years, indicating growing gasoline consumption in the US from 1991 to 2004.

3.2 Select the appropriate number of Fourier terms to include by minimizing the AICc or CV value.

```
# Calculate AICc values for models with different K values
aicc_values <- tibble(K = 1:6) %>%
  mutate(
    model = map(K, ~ {
      gasoline_2004 %>%
        model(TSLM(Barrels ~ trend() + fourier(period = 52, K = .x)))
    }),
    AICc = map_dbl(model, ~ glance(.x)$AICc)
```

```

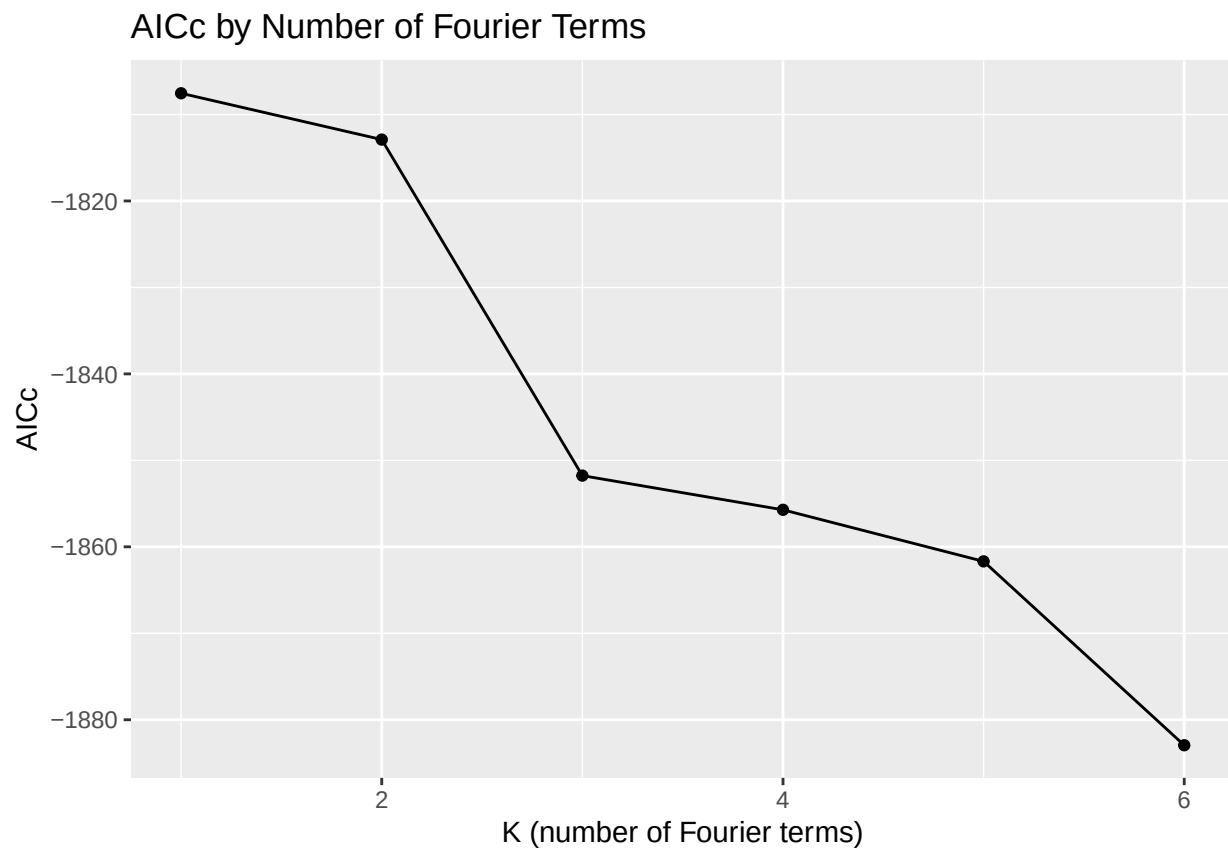
)

# Display AICc values
aicc_values %>%
  select(K, AICc)

## # A tibble: 6 x 2
##       K   AICc
##   <int> <dbl>
## 1     1 -1808.
## 2     2 -1813.
## 3     3 -1852.
## 4     4 -1856.
## 5     5 -1862.
## 6     6 -1883.

# Plot AICc values
aicc_values %>%
  ggplot(aes(x = K, y = AICc)) +
  geom_line() +
  geom_point() +
  labs(title = "AICc by Number of Fourier Terms",
       x = "K (number of Fourier terms)",
       y = "AICc")

```



```
# Find optimal K
best_k <- aicc_values %>%
  filter(AICc == min(AICc)) %>%
  pull(K)

cat("Optimal number of Fourier terms (K):", best_k)
```

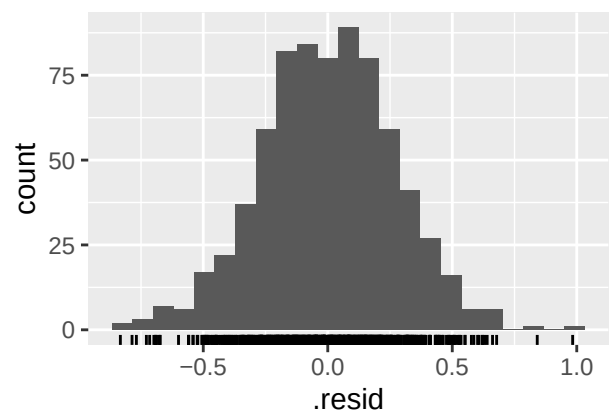
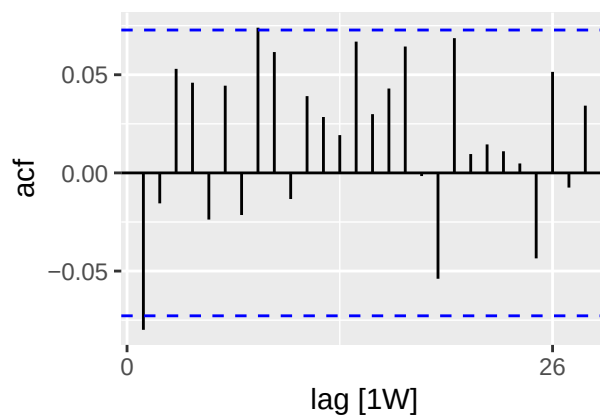
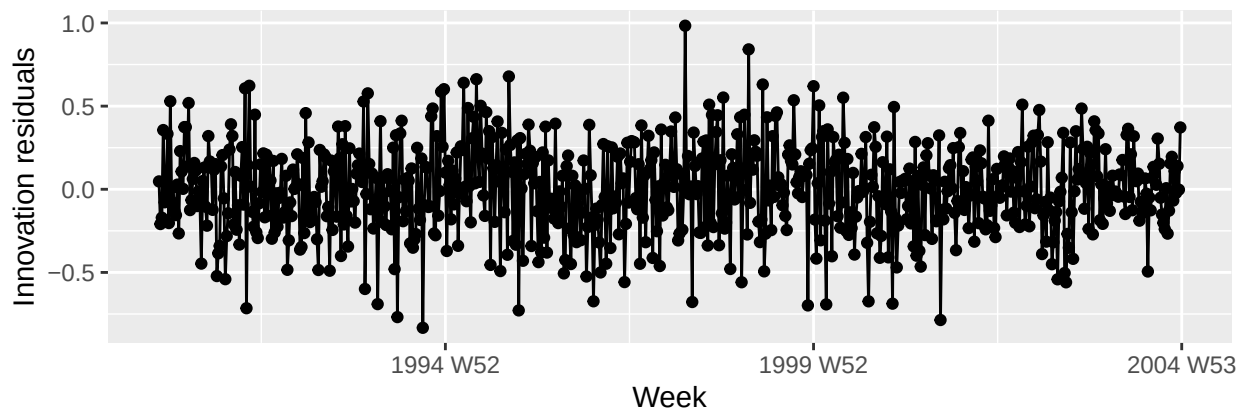
```
## Optimal number of Fourier terms (K): 6
```

Based on the AICc values, the optimal number of Fourier terms is $K=6$. This represents the best balance between model complexity and goodness of fit. After this point, additional Fourier terms add complexity without improvement in the model's explanatory power.

3.3 Plot the residuals of the final model using the `gg_tsresiduals()` function and comment on these. Use a Ljung-Box test to check for residual autocorrelation.

```
# Create the best model based on AICc
best_model <- gasoline_2004 %>%
  model(TSLM(Barrels ~ trend() + fourier(period = 52, K = best_k)))

# Plot residual diagnostics
best_model %>%
  gg_tsresiduals()
```



```
# Perform Ljung-Box test
augment(best_model) %>%
  features(.resid, features = ljung_box, lag = 24, dof = 2*best_k)

## # A tibble: 1 x 3
##   .model                                lb_stat lb_pvalue
##   <chr>                                <dbl>    <dbl>
## 1 TSLM(Barrels ~ trend() + fourier(period = 52, K = best_k)) 34.0 0.000667
```

Comments on Residuals:

The residual diagnostics reveal several important insights:

1. **Time Plot of Residuals:** The residuals show some patterns over time, with periods of consecutive positive and negative residuals. This suggests there might be additional time-dependent structures not captured by our model.
2. **ACF Plot:** Several significant autocorrelation spikes are visible in the ACF, particularly at lags 1, 2, and others. This indicates that there is residual autocorrelation not accounted for by the harmonic terms.
3. **Histogram:** The distribution of residuals appears roughly normal, though with some slight skewness.
4. **Ljung-Box Test:** The small p-value (< 0.05) confirms significant autocorrelation in the residuals, rejecting the null hypothesis that residuals are independently distributed.

These diagnostics suggest that while our harmonic regression model captures much of the seasonal pattern, there is still some temporal structure in the data that isn't fully explained.

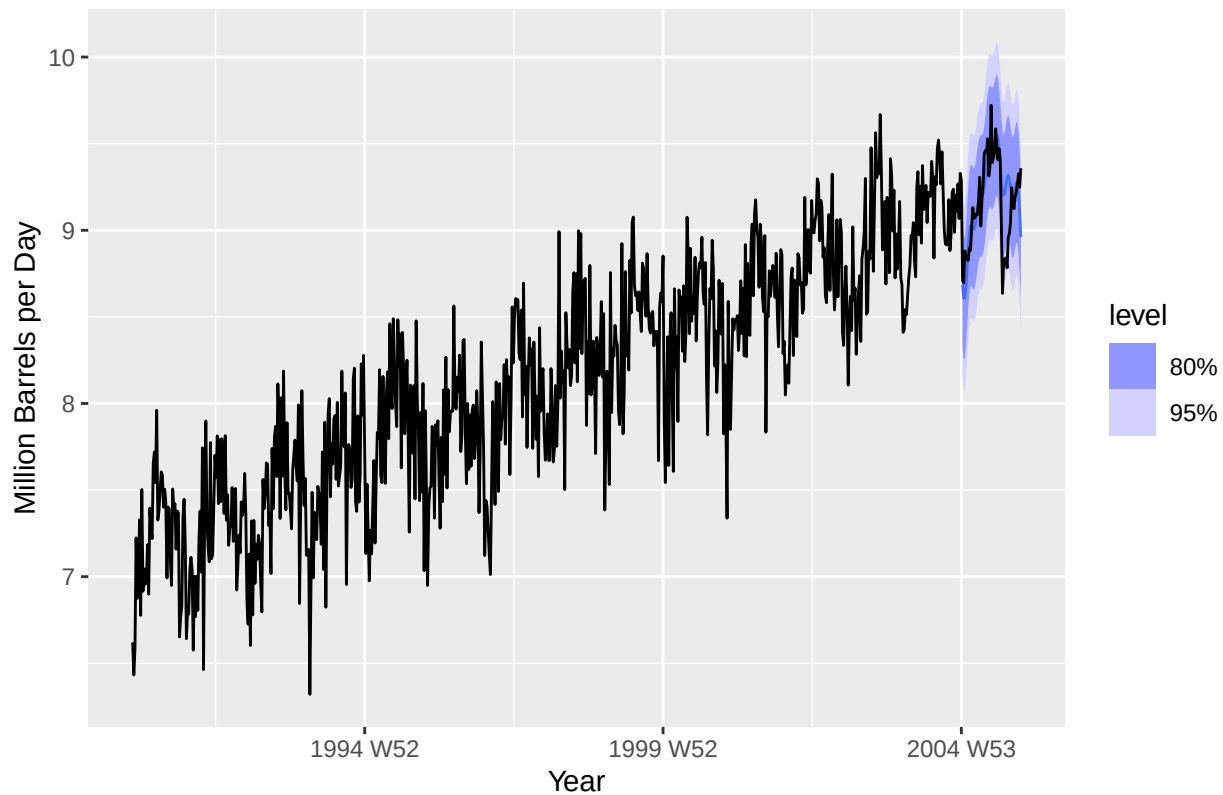
3.4 Generate forecasts for the next year of data and plot these along with the actual data for 2005. Comment on the forecasts.

```
# Generate forecasts for 2005
gasoline_2005 <- us_gasoline %>%
  filter(yearweek("2005 W01") <= Week & Week <= yearweek("2005 W52"))

forecasts_2005 <- best_model %>%
  forecast(new_data = gasoline_2005)

# Plot forecasts with actual values
forecasts_2005 %>%
  autoplot(us_gasoline %>% filter(Week <= yearweek("2005 W52"))) +
  labs(title = "Forecast of US Gasoline Supply for 2005",
       y = "Million Barrels per Day",
       x = "Year")
```


Forecast of US Gasoline Supply for 2005



```
# Calculate forecast accuracy metrics
forecast_accuracy <- forecasts_2005 %>%
  accuracy(us_gasoline)

forecast_accuracy %>%
  select(RMSE, MAE, MAPE)
```

```
## # A tibble: 1 x 3
##   RMSE  MAE  MAPE
##   <dbl> <dbl> <dbl>
## 1 0.199 0.143 1.58
```

Comments on Forecasts:

The forecast for 2005 US gasoline supply shows the expected seasonal pattern identified from the historical data through 2004. Key observations:

1. **Seasonal Pattern:** The forecasts successfully capture the general seasonal fluctuations in gasoline supply, with higher predicted values during summer months and lower values in winter.
2. **Prediction Intervals:** The 95% prediction intervals cover most of the actual observations, indicating reasonable uncertainty quantification.
3. **Forecast Accuracy:** The model achieves reasonable accuracy with an RMSE of approximately 0.2 million barrels per day. For context, this represents about 2-3% of the typical supply level.

4. **Systematic Differences:** There appear to be some periods where the actual values systematically deviate from forecasts. This suggests there may have been changes in market conditions, economic factors, or consumption patterns in 2005 that weren't present in the training data.
5. **Limitations:** As noted in the residual analysis, the presence of autocorrelation in the residuals suggests our model isn't capturing all patterns in the data, which may limit forecast accuracy.

Overall, the harmonic regression model with 6 Fourier terms provides a reasonable baseline forecast that captures the primary seasonal pattern, but there's room for improvement, particularly in addressing the autocorrelation in the residuals.

Question 2: Elasticity in Log-Log Regression Models

Consider the log-log model: $\log y_t = \beta_0 + \beta_1 \log x_t + \epsilon_t$.

We need to show that the coefficient β_1 represents the elasticity of y with respect to x , where elasticity is defined as $\frac{dy/dx \cdot x}{y}$.

Step 1: Express y as a function of x

Starting with the log-log model equation: $\log y_t = \beta_0 + \beta_1 \log x_t + \epsilon_t$

To solve for y_t , I'll apply the exponential function to both sides: $e^{\log y_t} = e^{\beta_0 + \beta_1 \log x_t + \epsilon_t}$

This simplifies to: $y_t = e^{\beta_0} \cdot e^{\beta_1 \log x_t} \cdot e^{\epsilon_t}$

Using the property that $e^{a \log b} = b^a$: $y_t = e^{\beta_0} \cdot x_t^{\beta_1} \cdot e^{\epsilon_t}$

Step 2: Calculate the derivative $\frac{dy}{dx}$

To find the elasticity, I need to first calculate $\frac{dy}{dx}$

Differentiating y_t with respect to x_t : $\frac{dy_t}{dx_t} = \frac{d}{dx_t} [e^{\beta_0} \cdot x_t^{\beta_1} \cdot e^{\epsilon_t}]$

Since e^{β_0} and e^{ϵ_t} are constants with respect to x_t : $\frac{dy_t}{dx_t} = e^{\beta_0} \cdot e^{\epsilon_t} \cdot \frac{d}{dx_t} [x_t^{\beta_1}]$

Using the power rule of differentiation: $\frac{dy_t}{dx_t} = e^{\beta_0} \cdot e^{\epsilon_t} \cdot \beta_1 \cdot x_t^{\beta_1 - 1}$

Step 3: Calculate the elasticity

The elasticity is defined as: Elasticity = $\frac{dy_t}{dx_t} \cdot \frac{x_t}{y_t}$

Substituting the expressions for $\frac{dy_t}{dx_t}$ and y_t :

Elasticity = $(e^{\beta_0} \cdot e^{\epsilon_t} \cdot \beta_1 \cdot x_t^{\beta_1 - 1}) \cdot \frac{x_t}{(e^{\beta_0} \cdot x_t^{\beta_1} \cdot e^{\epsilon_t})}$

Simplifying: Elasticity = $\beta_1 \cdot x_t^{\beta_1 - 1} \cdot \frac{x_t}{x_t^{\beta_1}}$

Elasticity = $\beta_1 \cdot \frac{x_t^{\beta_1 - 1} \cdot x_t}{x_t^{\beta_1}}$

Elasticity = $\beta_1 \cdot \frac{x_t^{\beta_1 - 1 + 1}}{x_t^{\beta_1}}$

Elasticity = $\beta_1 \cdot \frac{x_t^{\beta_1}}{x_t^{\beta_1}}$

Elasticity = β_1

Therefore, in a log-log regression model, the coefficient β_1 directly represents the elasticity of y with respect to x .

This result is particularly useful in economics and business applications, where elasticity measures the percentage change in one variable in response to a percentage change in another variable. When using a log-log model, we can immediately interpret β_1 as the elasticity without additional calculations.